BACSE202 - Database Systems

File Structures: Operations on Files - Files of Unordered and Ordered Record

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

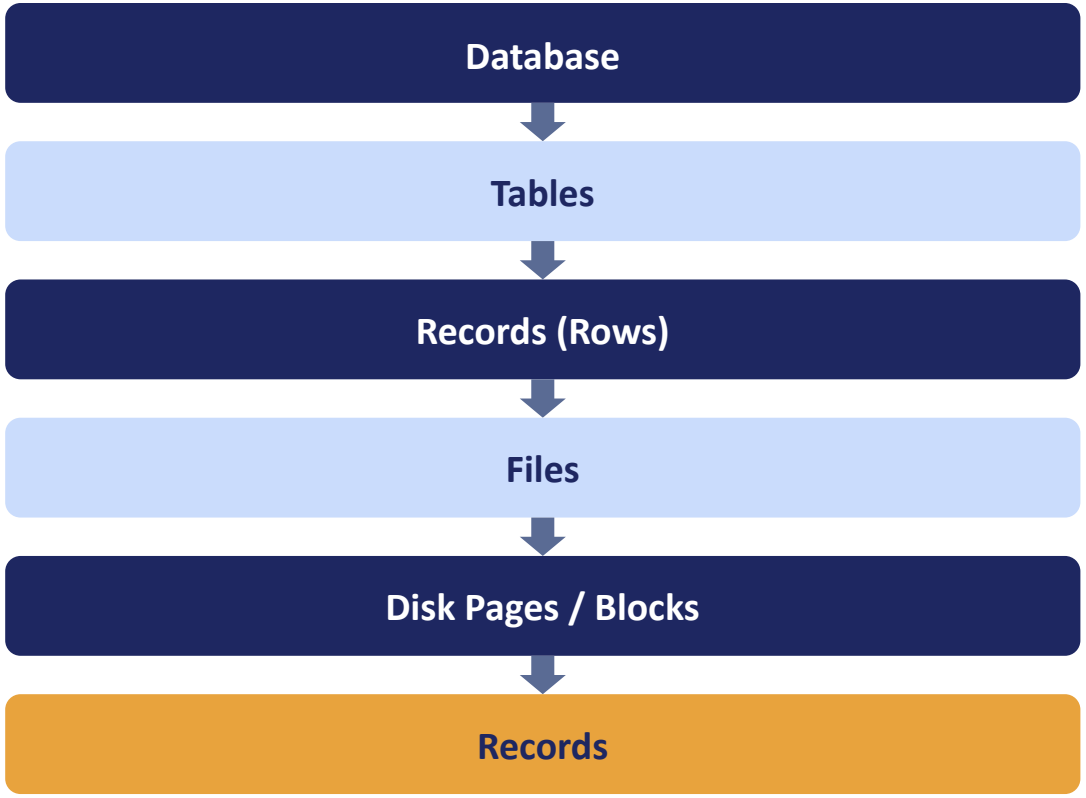
Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Where Does a Database Store Its Records?

A single table can hold millions of records — the DBMS needs a physical plan for arranging them on secondary storage.



"If 1 million student records are stored on disk, how should the DBMS arrange them so that searching is fast?"

1

PART I

Operations on Files

- What is a file, record, and field?
- Why file organization matters for performance
- The core operations: open, read, find, insert, delete, update, close
- Search as the operation most shaped by organization

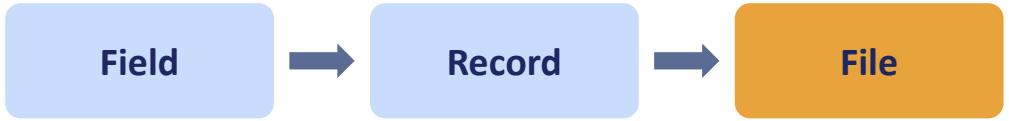
DEFINITIONS

What Is a File?

A file is a collection of related records stored on secondary storage.

STUDENT file

Student_ID	Name	Dept	CGPA
101	Arun	CSE	8.2
102	Ravi	IT	7.8
103	Kumar	CSE	9.1



KEY MAPPING

Each row → **A record**

Each attribute → **A field**

Complete collection → **A file**

THE CORE PROBLEM

Why File Organization Matters

10,00,000

student records to manage

WE NEED TO PERFORM

- Find a record
- Insert a record
- Delete a record
- Modify a record
- Read all records

There is no single file organization that is best for every operation.

A structure that makes searching fast may make insertion expensive.

The way records are arranged on disk determines the cost of every operation above.

THE TOOLKIT

Basic Operations on Files

Every file organization must support the same seven operations — organization changes their cost, not their existence.

1

Open

Prepare the file for use

2

Read / Retrieve

Bring records into memory

3

Find / Search

Locate a specific record

4

Insert

Add a new record

5

Delete

Remove an existing record

6

Modify / Update

Change a record's values

7

Close

Release the file

THE DECIDING OPERATION

Search: Two Ways to Find a Record

`Query: Find Student_ID = 1057`

METHOD 1 · SCAN

`101 → 102 → 103 → … → 1057`

Walk through records one by one until a match is found.

Cost grows with file size

METHOD 2 · ORGANIZED ACCESS

`Student_ID → Storage location → Record`

Use the file's organization (or an index) to jump close to the answer.

Cost depends on the structure chosen

Key question: *How the records are organized determines how efficiently a record can be found.*

Insert, Delete and Update

INSERT

```
INSERT Student 110
```

Add a new record to the file.

DELETE

```
DELETE Student 105
```

Remove an existing record.

UPDATE

```
UPDATE Student 105
SET CGPA = 8.7
```

Change an existing record's values.

Important: The cost of insert, delete, and update depends heavily on the file's organization.

2

PART II

Files of Unordered Records

- Also called Heap File organization
- Records placed wherever space is available
- Insertion is simple and fast
- Searching and deletion trade-offs

HEAP ORGANIZATION

Unordered Files: No Particular Order



Records stored in no particular order — Student_ID need not be sorted.

MAIN IDEA

Put the record wherever there is available space.

INSERTION EXAMPLE

Insert 103 — it does not need to sit between 101 and 105.



↑ *appended in the first free slot*

ADVANTAGE

Insertion is simple and fast.

Because no ordering needs to be maintained, a new record can go into any available space — no shifting, no reorganizing.

TRADE-OFFS

Searching & Deletion in Unordered Files

SEARCH

`Find Student_ID = 108`

101 → 105 → 143 → 112 → 108

Equality search may need to examine every record — the match could be anywhere.

Favors insertion, not searching

DELETION

`Delete 108 from: 101 | 105 | 108 | 112 | 103`

101 | 105 | EMPTY | 112 | 103

The freed slot can simply be reused later — no ordering to preserve.

Relatively simple

Unordered file favors insertion but does not favor searching.

SUMMING UP

Characteristics of Unordered Files

Operation	Performance
Insert	**Very good**
Search	**Poor for large files**
Delete	**Relatively easy**
Update	Depends on search
Sequential processing	Possible
Maintaining order	Not required

"Unordered = Easy to put, difficult to find."

3

PART III

Files of Ordered Records

- Also called Sequential File organization
- Records stored according to an ordering field
- Searching and sequential access become efficient
- Insertion and deletion get more expensive

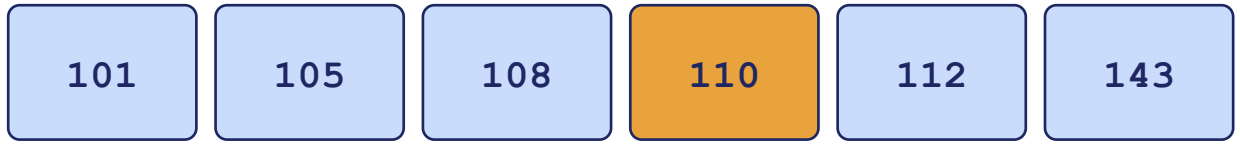
SEQUENTIAL ORGANIZATION

Ordered Files: Sorted by a Key Field

Records are stored according to the value of an ordering field — e.g. Student_ID, Employee_ID, Account_Number, Roll_Number.



NOW INSERT 110 — IT MUST LOGICALLY APPEAR BETWEEN 108 AND 112

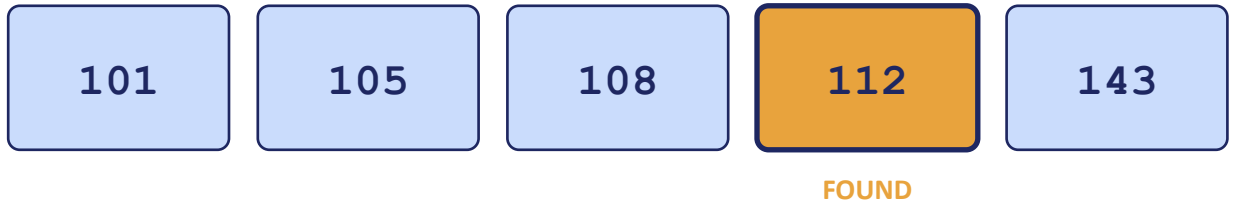


Important consequence: The system must maintain the ordering, so insertion becomes more expensive — later records may need to shift.

WHY WE SORT

Searching in Ordered Files

`Find Student_ID = 112`



BEYOND SCANNING

Because records are ordered, we can use the ordering to eliminate large portions of the search space — instead of scanning everything one by one.

Ordered files favor searching and sequential processing.

Insertion, Overflow & Deletion

INSERTION & OVERFLOW

`Insert 110 into: 101 | 105 | 108 | 112 | 143`

→ **101 | 105 | 108 | 110 | 112 | 143**

Problem: what if there is no free space at the required location?

The DBMS may need overflow areas / files, and periodic reorganization — an important lead-in to later topics.

DELETION

`Delete 108 from: 101 | 105 | 108 | 112 | 143`

→ **101 | 105 | 112 | 143**

With large files, repeated insertions and deletions create unused spaces and overflow records.

Maintaining an ordered file can become costly over time.

4

PART IV

Comparison & Beyond

- Unordered vs. Ordered — side-by-side view
- A simple real-life analogy for both structures
- Why the workload decides the best organization
- The bridge to Indexing and Hashing

HEAD TO HEAD

Unordered vs. Ordered Files

Feature	Unordered File	Ordered File
Record order	No particular order	Sorted by ordering field
Insertion	**Easy**	More expensive
Search	Generally slower	**More efficient**
Deletion	Easy	More complicated
Sequential access	Possible	**Very efficient**
Maintaining order	Not required	Required
Best suited for	Frequent insertions	Frequent retrieval / sequential processing

Unordered files optimize insertion; ordered files optimize retrieval and sequential processing.

A Simple Real-Life Analogy

U Unordered File = Toy Box

You put a toy wherever there is space.

Putting a new toy: **Easy**

Finding a particular toy: **Difficult**

O Ordered File = Bookshelf

Books are arranged alphabetically.



Finding a book: **Easier**

Placing a new book: **More work**

Why Do We Need Different File Organizations?

SCENARIO 1

A transaction system receives thousands of new records every minute.

→ **Unordered file**

Insertion speed is what matters most.

SCENARIO 2

A system frequently retrieves records and processes them in sorted order.

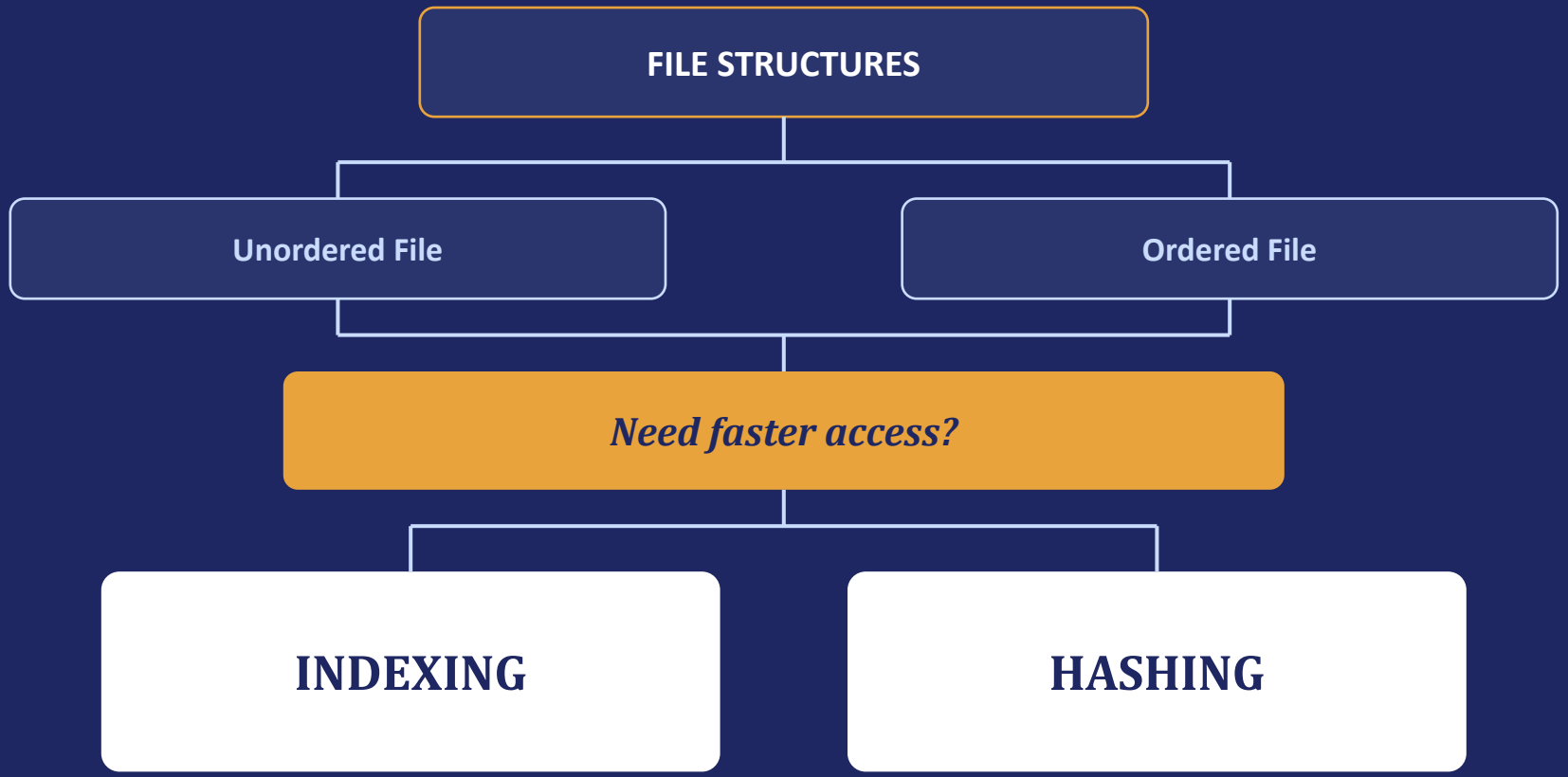
→ **Ordered file**

Retrieval and sequential processing matter most.

Key lesson: *The best file organization depends on the workload.*

THE BRIDGE

Connection to Indexing and Hashing



How can we find records faster without scanning the entire file? Indexing and hashing are the answers we explore next.

Thank You

BACSE202 - Database Systems

Hashing Techniques: Static and Dynamic

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Why Hashing?

In huge databases it is very impractical to search all the index values to get the desired data as searching all the index values through all the levels is not an efficient way to reach destination data block to retrieve the desired data.

So we need an efficient technique

What is Hashing?

Hashing is a method for calculating the direct position of an information record on the disk without the use of an index structure.

It is an effective technique with less search time

It uses a hash function (a mathematical function) to find the exact location of a record in the minimum amount of time.

Some important Terminologies

Data buckets are the *memory locations* where the actual data records are stored.

Hash function - A mathematical function that returns the value of the address of the desired data block. Mostly, it *uses the primary key* to generate the address of a data block.

Hash index - It denotes the *address of a data block* generated by the hash function.

Linear Probing - It is used when a record already exists on the memory location calculated by Hash Function. It linearly searches for an empty bucket until it is found.

Some important Terminologies

Quadratic Probing - It is similar to linear probing with the only difference as it uses an *arbitrary polynomial function* to search for the next empty bucket instead of searching linearly.

Bucket overflow - The condition when the memory address generated by the hash function is not empty(*i.e.* it already contains some data record) is known as bucket overflow

Properties of Hashing in DBMS

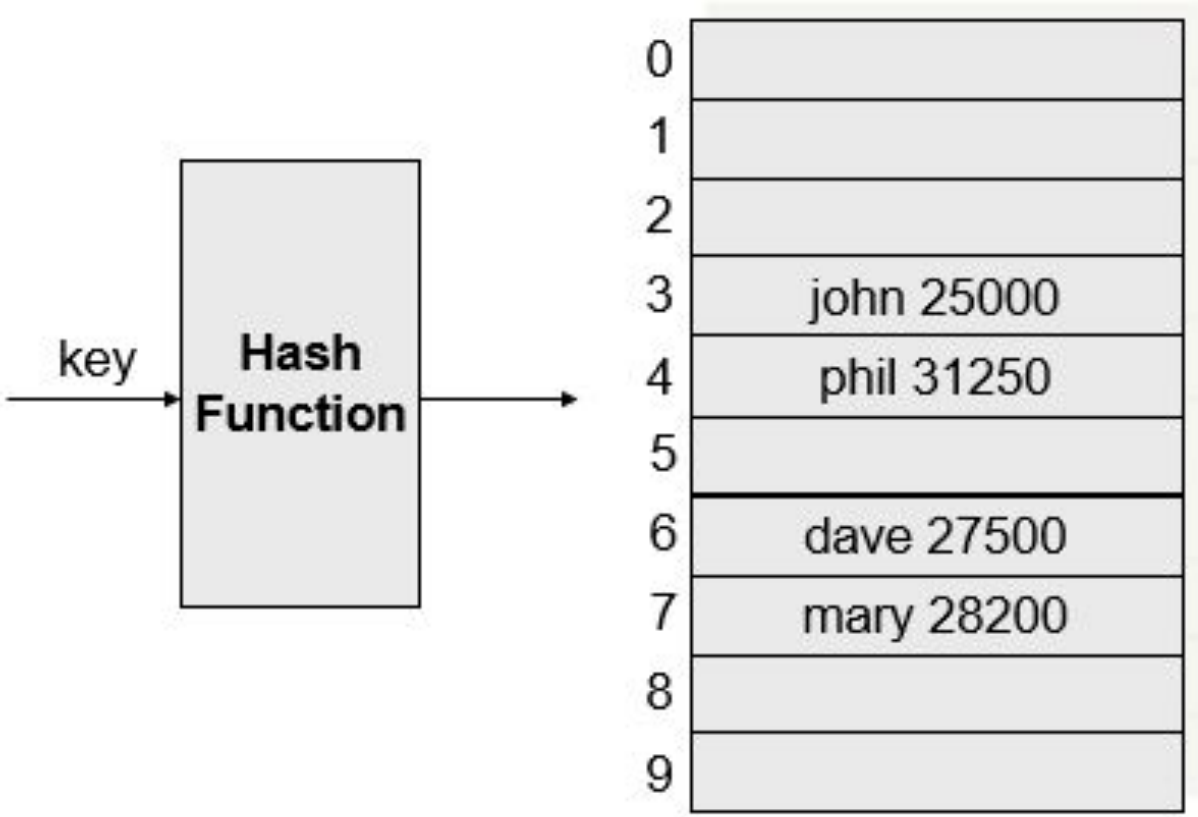
- Data is kept in data blocks whose addresses are produced using the hashing function.
- Data buckets or data blocks are the memory locations where these records are stored.
- A hash function can produce the address from any column value.
- The primary key is frequently used by the hash function to generate the data block's address.
- To every complex mathematical function, a hash function is a basic mathematical function.
- The primary key can also be considered as the data block's address, i.e. each row with the same address as a primary key contained in the data block.

Hash Function

- **The hash function:**
  - must be simple to compute.
  - must **distribute the keys evenly** among the cells.

- **If we know which keys will occur in advance we can write *perfect* hash functions, but we don't.**

Example


key
Hash Function
0
1
2
3
john 25000
4
phil 31250
5
6
dave 27500
7
mary 28200
8
9

Properties of Good Hash Functions

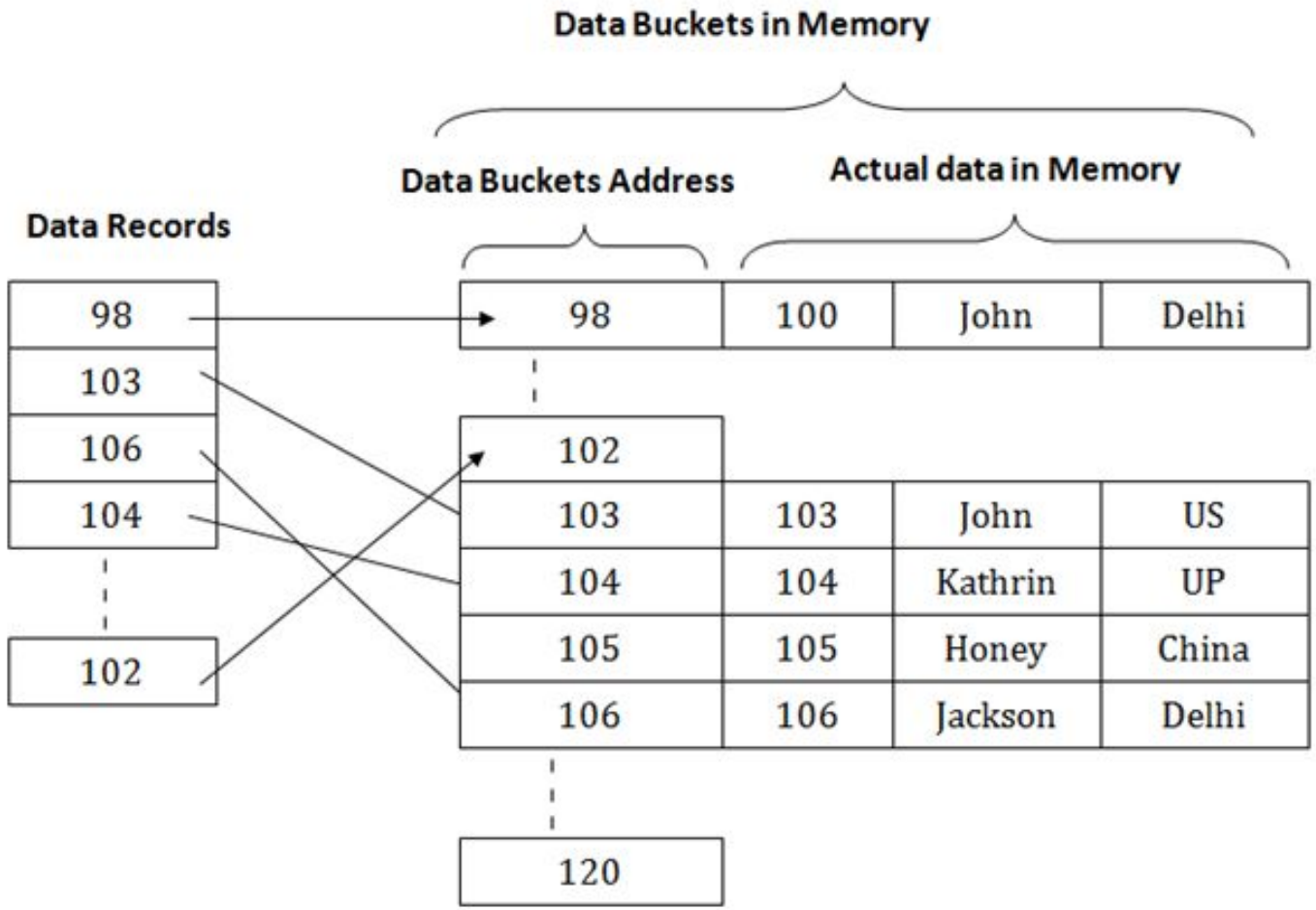
- **Must return number 0, …, table size**
- **Should be efficiently computable – O(1) time**
- **Should not waste space unnecessarily**
  - For every index, there is at least one key that hashes to it
  - **Load factor lambda λ = (number of keys / Table Size)**
- **Should minimize collisions**

  = different keys hashing to same index

  - **Hash(x) = x % TableSize**

Hashing

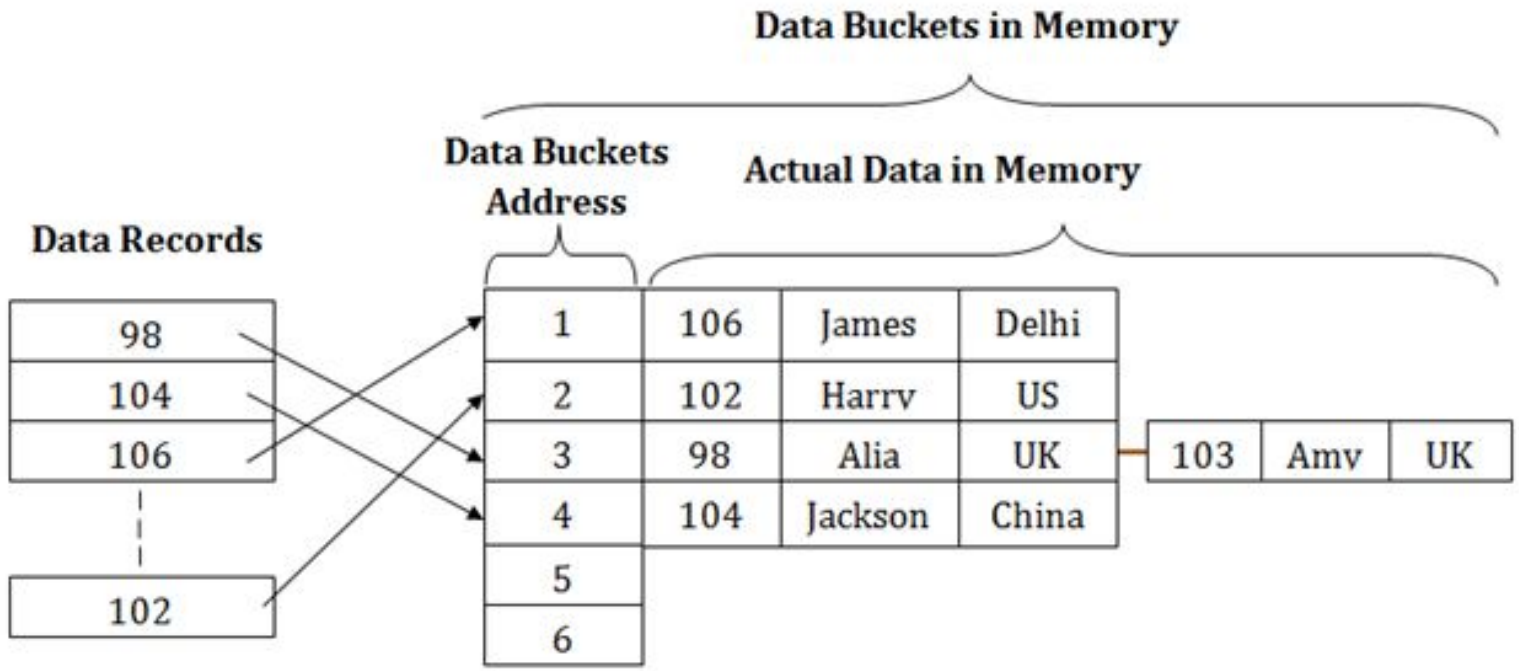
The diagram shows data block addresses same as primary key value.



Hashing

The hash function could alternatively be a simple mathematical function, such as exponential, mod, cos, sin, and so on.

For example, the **mod (5) hash function** is used to find the data block's address in the below scenario, here the primary keys are hashed with the mod (5) function, yielding 3, 3, 1, 4, and 2, respectively, and records are saved at those data block locations.



Hashing

- Process of **mapping** a key value to a position in a table

Another Example: 7, 18 , 41 , 34

7 % 10 = **7**

18 % 10 = **8**

41 % 10 = **1**

34 % 10 = **4**

0	
1	**41**
2	
3	
4	**34**
5	
6	
7	**7**
8	**18**
9	

Hashing

- Hash Table is an array of some **fixed size**

- Search is performed based on the **key**

- Each key is mapped into some position in the range **0 to table size - 1**

Drawbacks:

- **Waste too much space**, if universe is too **large** compared with actual number of elements to be stored

- **Collision** {i.e. Two keys may hash to same slot}

Example: To insert elements 80 , 40 , 65 , 24 , 58 , 35 , 98

Table Size = 11

80 % 11 = **3**

40 % 11 = **7**

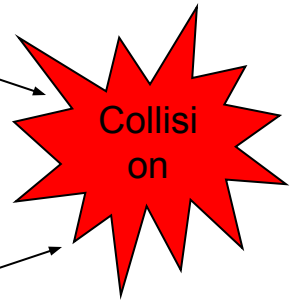
65 % 11 = **10**

24 % 11 = **2**

58 % 11 = **3**

35 % 11 = **2**

98 % 11 = **10**



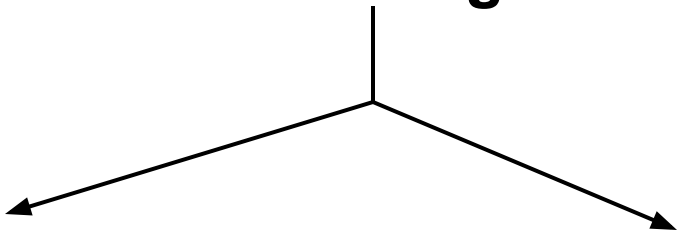
0	
1	
2	**24**
3	**80**
4	
5	
6	
7	**40**
8	
9	
10	**65**

Collision in Hashing

- A collision occurs when the hash field value of a record that is being inserted hashes to an address that already contains a different record.
- In this situation, we must insert the new record in some other position, since its hash address is occupied.
- The process of finding another position is called collision resolution.

- There are numerous methods for collision resolution, including the following:
- Open Addressing / Closed Hashing
- Chaining / Separate chaining / Open Hashing
- Multiple Hashing

Collision Handling Strategies



Open Hashing

Separate Chaining

Open Addressing

Closed Hashing

- Linear Probing
- Quadratic Probing
- Double Hashing
- Rehashing

Hashing Types

1. Static Hashing
2. Dynamic Hashing

HASHING IN DATABASE AND TYPES

Hashing is a technique that maps a search key to the address of a data block (bucket) using a hash function (direct access – no need to search an index).

HOW HASHING WORKS

Search Key (e.g., StudentID): 105 → Hash Function h(key) → Bucket Address (e.g., 5) → Data Bucket

Bucket 5
105, Ram
205, Sita
305, John
...

h(key) = key % N
where N = number of buckets

EXAMPLE

Let N = 10 (Buckets 0 to 9) and h(key) = key % 10

Key	h(key) = key % 10
23	3
43	3
15	5
18	8
34	4

Hash Table (Buckets)

0	-
1	-
2	-
3	23, 43 ← Collision
4	34
5	15
6	-
7	-
8	18
9	-

Collision

IMPORTANT TERMINOLOGIES

- **Data Bucket** : Memory location where actual data records are stored.
- **Hash Function** : A mathematical function that returns the address of the desired data bucket (usually uses primary key).
- **Hash Index** : The address (bucket number) generated by the hash function.
- **Collision** : When two different keys map to the same bucket.
- **Bucket Overflow** : When a bucket is full and cannot accommodate more records.

PROPERTIES OF GOOD HASH FUNCTION

- Must return a number in the range 0 to N-1 (N = table size).
- Should be **simple and fast** to compute → O(1) time.
- Should distribute keys evenly among buckets.
- Should minimize collisions.
- **Load Factor (λ)** = Number of keys / Table Size. (Ideal λ is between 0.5 to 0.75)

COLLISION – WHAT & WHY?

When different keys map to the same bucket address.

23 % 10 = 3
43 % 10 = 3
} Collision

WHY COLLISION HAPPENS?

Because the number of possible keys is usually much larger than the number of buckets.

Large Key Space → Limited Buckets

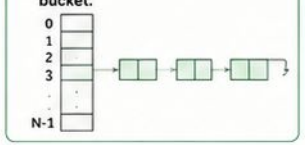
TYPES OF HASHING

1. STATIC HASHING

Number of buckets is fixed. Used when the size of the database is relatively stable.

(A) SEPARATE CHAINING

If collision occurs, link the new record in a list (chain) at that bucket.



(B) OPEN ADDRESSING

If collision occurs, probe other buckets in the table to find an empty slot.

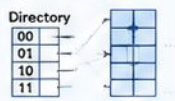
Linear Probing | Quadratic Probing | Double Hashing

2. DYNAMIC HASHING

Number of buckets grows or shrinks dynamically with the data.

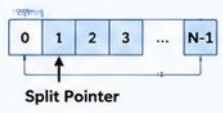
(A) EXTENDIBLE HASHING

Directory based technique. When a bucket overflows, the directory is extended (using more bits).



(B) LINEAR HASHING

No directory. Buckets are added one at a time using a simple linear order.



COMPARISON

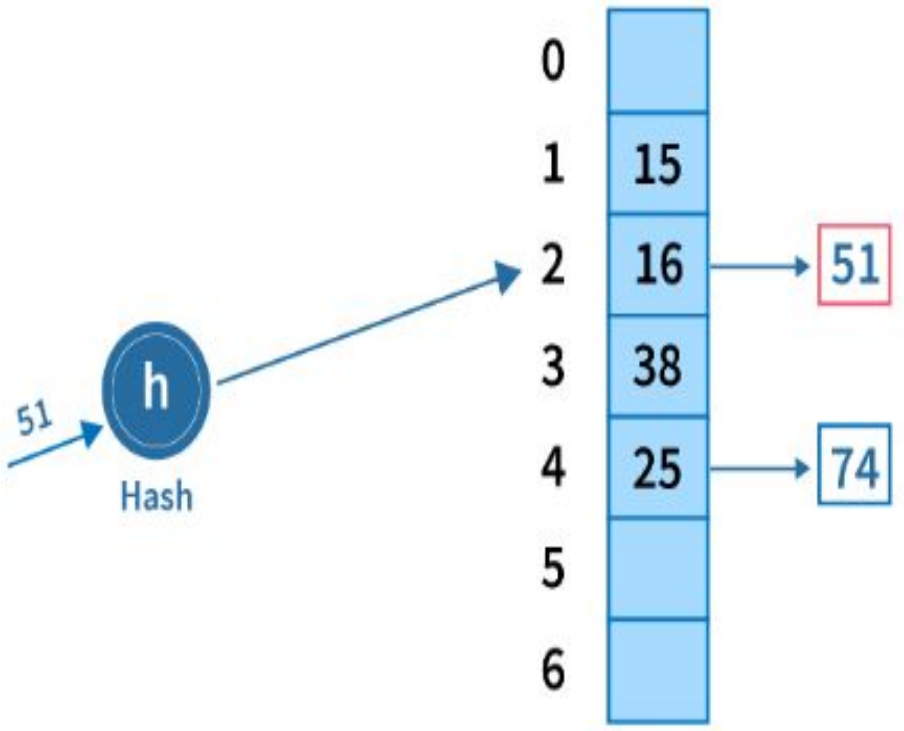
Feature	Static Hashing	Dynamic Hashing
Number of Buckets	Fixed	Can grow or shrink
Overflow Handling	Using chains or probing	By adding buckets
Performance	May degrade when table is full	Better as it adapts to data size
Complexity	Simple	More complex
Space Usage	May waste space	More efficient

Key Takeaway: Hashing provides very fast access (O(1) average time) by mapping keys directly to buckets. Choose the right hashing type and collision handling strategy based on data size and growth.

Types of Static Hashing

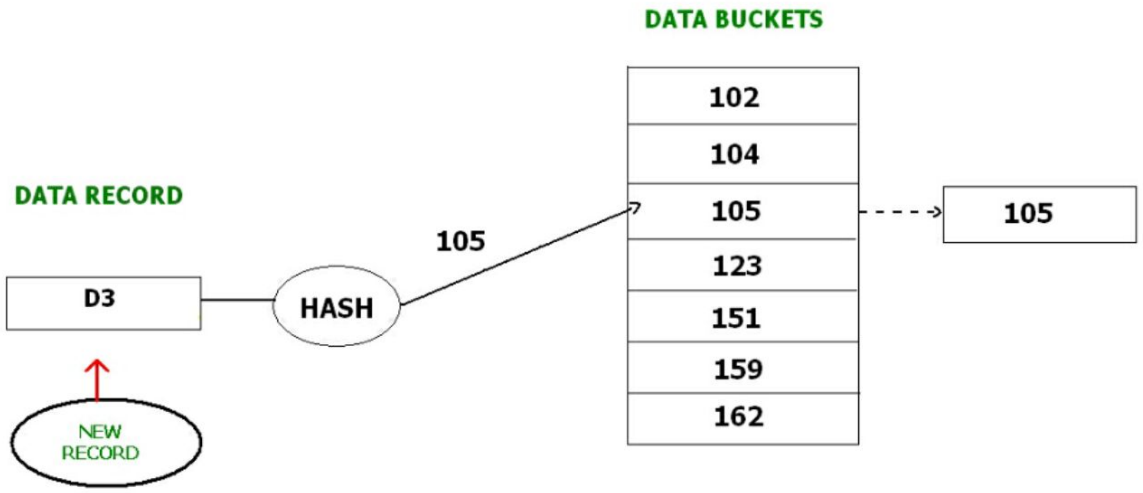
1. Open Hashing / Separate Chaining - In open hashing, the collision condition is handled by linking the new record after the previous one, due to which is also termed as "Hashing with separate chaining".

Here, when buckets are full, a new bucket is allocated for the same hash and result are linked after the previous one.

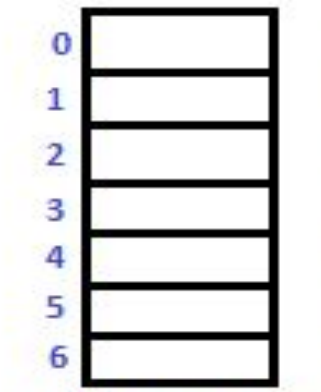


Separate Chaining

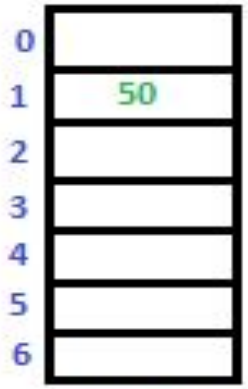
- In this method, a new data bucket is allocated with same address and is linked it after the full data bucket. This method is also known as Separate / overflow chaining.
- For example, we have to insert a new record D3 into the tables. The static hash function generates the data bucket address as 105. But this bucket is full to store the new data. In this case is a new data bucket is added at the end of 105 data bucket and is linked to it. Then new record D3 is inserted into the new bucket.



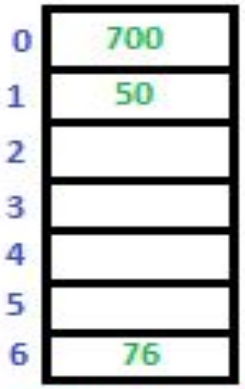
Example 2: Let us consider a simple hash function as "**key mod 7**" and sequence of keys as 50, 700, 76, 85, 92, 73, 101.



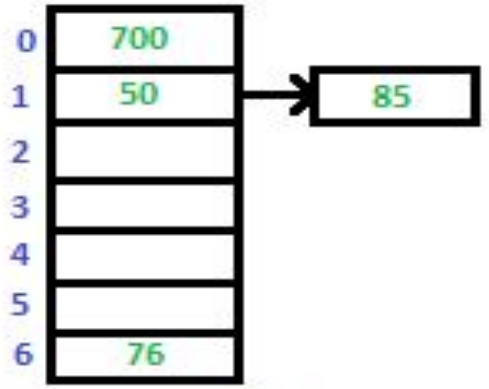
Initial Empty Table



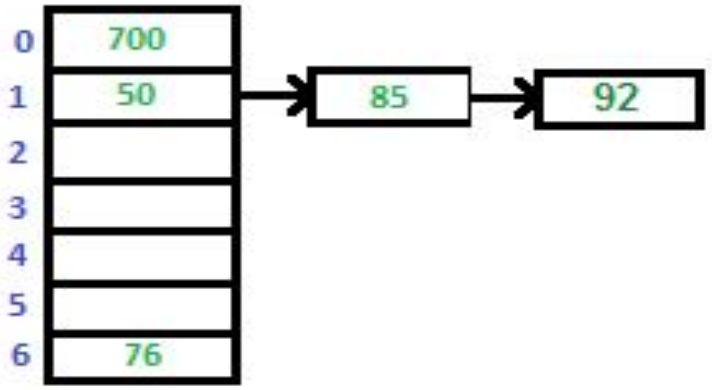
Insert 50



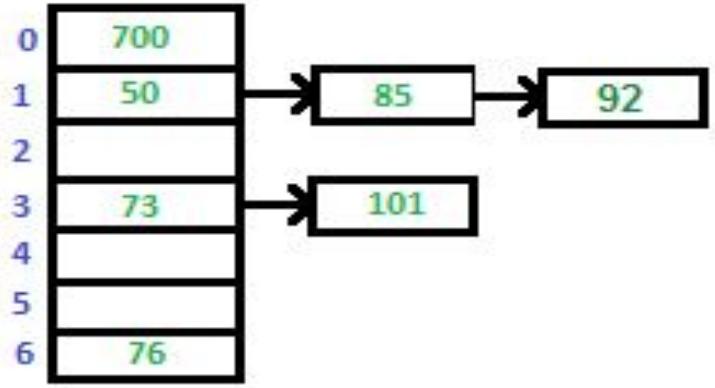
Insert 700 and 76



Insert 85: Collision Occurs, add to chain



Inser 92 Collision Occurs, add to chain



Insert 73 and 101

Advantages of Chaining

- **Simple to Implement**
- **Separate chaining is a very simple technique to implement compared to other data structures. Input elements are just added to the corresponding linked list which the input has been hashed to. No collision occurs in the table as the cells hold linked information.**
- **Hash Table Capacity not Limited**
- **There is no issue with the table being filled to capacity, as the table cells only hold linked information. New elements are just added to the corresponding chain.**
- **The hash table can be of theoretically infinite size**
- **The hash table will not get polluted**
- **When searching, it will never search irrelevant information.**

Disadvantages

- **Chaining has the disadvantage of using linked lists.**
  - Requires the implementation of a **second data structure.**

2. Static Hashing

- In static hashing, for a key *k*, hash function *h*(*x*) always *generates the same memory address*. For example, if *h*(x)=x%7 then for any *x*, the value will always be the same.
- Thus if we generate the address for a key 107, then *h*(*x*) will always return the same bucket address, *i.e.*, 2.
- There will be no change in the bucket address. Hence the number of buckets in the memory always remains constant.

- Collision Avoidance Methods: Linear Probing and Quadratic Probing

Operations in Static Hashing

- **Inserting a record: When a new record requires to be inserted into the table, you can generate an address for the new record using its hash key. When the address is generated, the record is automatically stored in that location.**

- **Searching: When you need to retrieve the record, the same hash function should be helpful to retrieve the address of the bucket where data should be stored.**

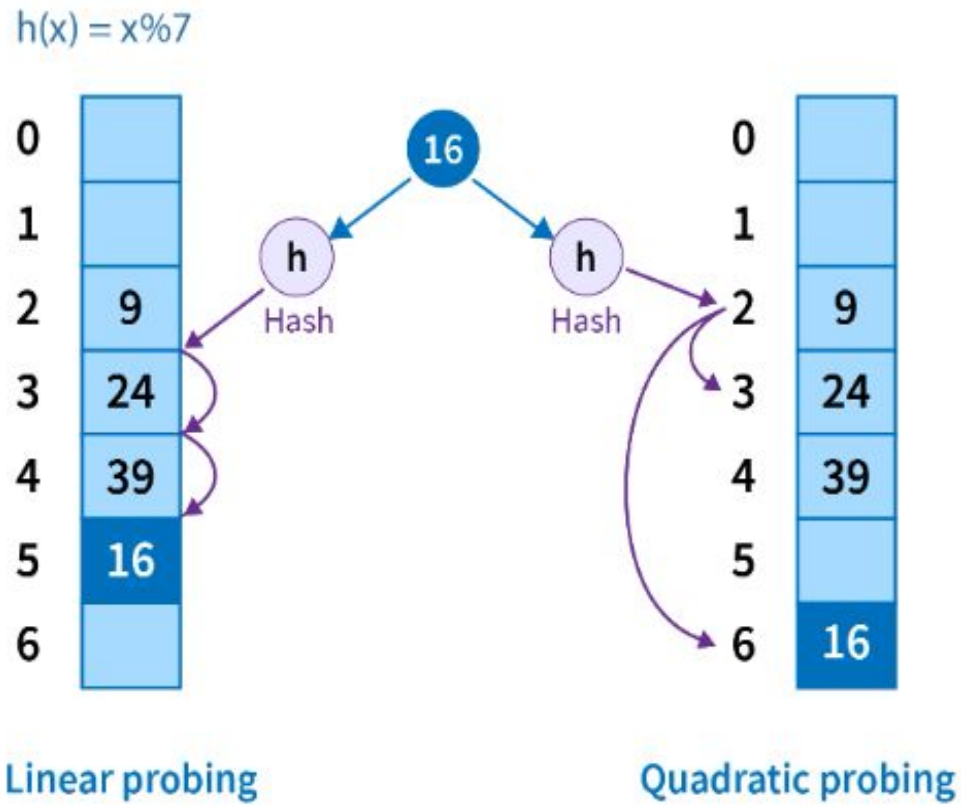
- **Delete a record: Using the hash function, you can first fetch the record which is you wants to delete. Then you can remove the records for that address in memory.**

Types of Static Hashing

1. **Open Addressing / Closed Hashing**

In open addressing, whenever a collision occurs, we probe for the next empty bucket to enter the new record.

The order of checking are majorly of two types -



Linear Probing

Example: 89 , 18 , 49 , 58 , 69

$$h(k) = k \bmod 10$$

89 % 10 = 9

18 % 10 = 8

49 % 10 = 9

58 % 10 = 8

69 % 10 = 9

0	49
1	58
2	69
3	
4	
5	
6	
7	
8	18
9	89

Linear Probing

- **Linear probing is a form of open addressing.**
- **In these schemes, each cell of a hash table stores a single key–value pair.**
- **When the hash function causes a collision by mapping a new key to a cell of the hash table that is already occupied by another key, linear probing searches the table for the closest following free location and inserts the new key there.**

$$h(x) = x \bmod \text{size of array}$$

if collision occurs

$$h(x) = [\, h(x) + f(i)] \bmod \text{size}$$

where f(i)= i, where i =0,1,2,3,…

Linear Probing

- **In linear probing, collisions are resolved by sequentially scanning an array (with wrap around) until an empty cell is found.**
  - i.e. *f* is a linear function of *i*, typically *f(i)* = *i*.

Drawbacks

- Clustering of element in particular location .The time taken to check will be more.
- Encounter **Primary Clustering** { blocks of contiguously occupied table entries}
- Many consecutive elements form groups and it starts taking time to find a free slot or to search for an element.
- If h(k) falls in Cluster, Cluster will **definitely grow in size by one**
- If **two Cluster** separated by one entry, inserting one key into cluster can merge two cluster, So cluster size increase drastically by single insertion
- Large Cluster are easy **targets for Collision**

Drawbacks….

- As long as table is big enough, a free cell can always be found, but the time to do so can get quite large.
- Any key that hashes into the cluster will require several attempts to resolve the collision, and then it will add to the cluster
- So to avoid this we go for quadrating probing

Quadratic Probing

- **h(x) =[h(x) +f(i)] mod Table_size**

  where f(i)= i^2, where i =0,1,2,3,…

- **Main Idea: Spread out the search for an empty slot – Increment by i^2 instead of i**
- **$h_i(X)$ = (Hash(X) + i^2) % *Table Size* , *i=0,1,2,3…..,***

  $h_0(X)$ = Hash(X) % Table Size

  $h_1(X)$ = [Hash(X) + 1] % Table Size

  $h_2(X)$ = [Hash(X) + 4] % Table Size

  $h_3(X)$ = [Hash(X) + 9] % Table Size

Quadratic Probing

Example: insert 14,8,21,2,7 with size m=7

insert(14)	insert(8)	insert(21)	insert(2)
14%7 = 0	8%7 = 1	21%7 =0	2%7 = 2
0: 14	0: 14	0: 14	0: 14
1:	1: 8	1: 8	1: 8
2:	2:	2:	2: 2
3:	3:	3:	3:
4:	4:	4: 21	4: 21
5:	5:	5:	5:
6:	6:	6:	6:
probes: 1	probes: 1	probes: 3	probes: 1

Algorithm

1. Get the key hash key k
2. Set counter j = 0
3. Compute hash function h[k] = k % SIZE
4. If hash table[h[k]] is empty
   (4.1) Insert key k at hash table[h[k]]
   (4.2) Stop
   Else
   (4.3) The key space at hash table[h[k]] is occupied, so we need to find the next available key space
   (4.4) Increment j
   (4.5) Compute new hash function h[k] = (k + j * j) % SIZE
   (4.6) Repeat Step 4 till j is equal to the SIZE of hash table
5. The hash table is full
6. Stop

Multiple Hashing / Double Hashing

Double hashing is a collision resolving technique in Open Addressed Hash tables. Double hashing uses the idea of applying a second hash function to key when a collision occurs.

This method applies a second hash function if the first results in a collision.

If another collision results, the program uses open addressing or applies a third hash function and then uses open addressing if necessary.

Double hashing can be done using :

(hash1(key) + i * hash2(key)) % TABLE_SIZE

Double Hashing

Here hash1() and hash2() are hash functions and TABLE_SIZE is size of hash table.
(We repeat by increasing i when collision occurs)

First Hash function **:** **hash1(key) = key % TABLE_SIZE**

Second Hash function **: hash2(key) = PRIME – (key % PRIME)**
where PRIME is a *prime smaller than the TABLE_SIZE.*

A good second Hash function is:
- It must never evaluate to zero
- Must make sure that all cells can be probed

Double Hashing

- **Idea: Spread out the search for an empty slot by using a second hash function**
  - *No primary or secondary clustering*
- **$h_i(X) = (Hash_1(X) + i*Hash_2(X))$ mod *Table Size***
  for i = 0, 1, 2, …
- **Good choice of $Hash_2(X)$ can guarantee does not get "stuck" as long as $\lambda < 1$**
  - Integer keys:
    $Hash_2(X) = R - (X \bmod R)$
    where R is a prime smaller than *Table Size*

An Example

Lets say, Hash1 (key) = key % 13

Hash2 (key) = 7 – (key % 7)

Hash1(19) = 19 % 13 = 6

Hash1(27) = 27 % 13 = 1

Hash1(36) = 36 % 13 = 10

Hash1(10) = 10 % 13 = 10

Hash2(10) = 7 – (10%7) = 4

(Hash1(10) + 1*Hash2(10))%13= 1

(Hash1(10) + 2*Hash2(10))%13= 5

Collision

Advantages:

Skip values give different search paths for keys that collide.

Disadvantage:

Delete cumbersome to implement

Home work : Hashing problem

Given the following input (3471, 1323, 6173, 4199, 4344, 9679,1989) and a hash function h(x)= x mod 10 show the resulting

A) Separate chaining hash table

B) Hash table using Linear Probing

C) Hash table using Quadratic Probing

D) Hash table with second Hash function

$$h_2(x) = 7 - (x \bmod 7)$$

Thank You

BACSE202 - Database Systems

Indexing: Single Level Indexing, Multi-Level Indexing, Dynamic Multilevel Indexing - B+ Tree Indexing – LSM Trees and its Variants

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Why Do We Need Indexes?

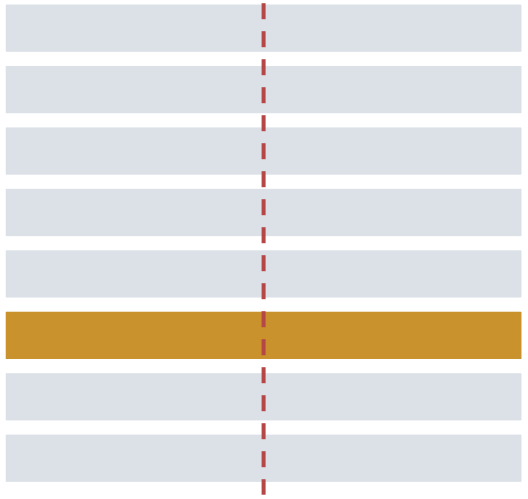
An index is an auxiliary access structure built on one or more fields of a file to speed up retrieval of records that satisfy a search condition.

- Without an index, the DBMS must perform a linear search, examining records one by one.
- An index trades a small amount of extra storage for a large reduction in the number of block accesses.
- Cost is commonly measured in number of disk block accesses, since disk I/O dominates query time.

Cost comparison – 100,000 records, 20 per block

Linear search (avg): ~2,500 block accesses
Multi-level index search: ~3–4 block accesses

Full Scan



scans every block until match found

With Index



jumps directly to the target block

Three Types of Ordered Single-Level Indexes

1

Primary Index

Built on the ordering key field of a file that is sorted on that field. One index entry per data block (sparse).

2

Clustering Index

Built on a non-key ordering field of a sorted file. One index entry per distinct value of the field (sparse).

3

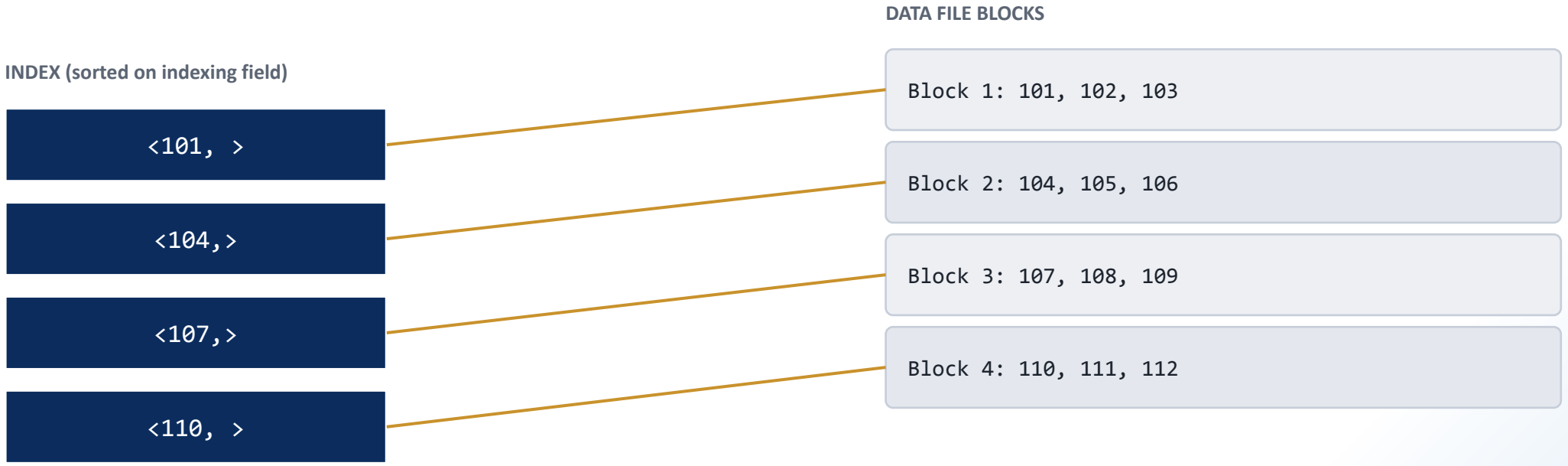
Secondary Index

Built on any non-ordering field (key or non-key). One index entry per record (dense) – file need not be sorted.

Sparse vs. Dense: A sparse index has one entry per data block (needs data file sorted); a dense index has one entry per record (works on any file, but is larger).

Primary Index – Sparse, on the Ordering Key

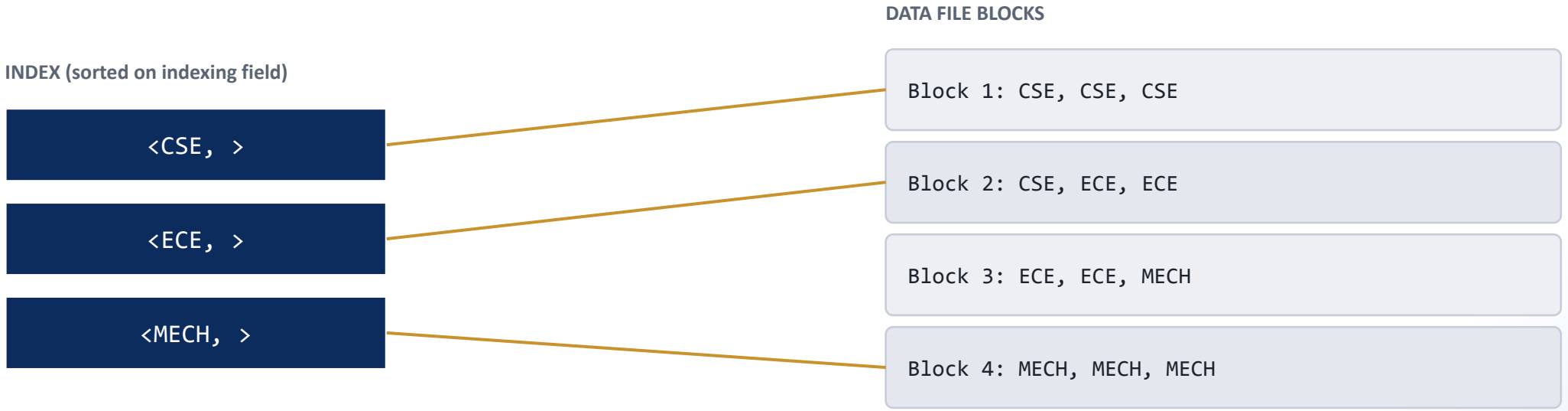
Data file is physically sorted on the primary key (e.g., Roll_No). The index stores one <key, block pointer> entry per data block – the block's first key.



Search cost: $O(\log_2 b_i)$ on the index (b_i index blocks) + 1 block access to fetch the data block — far cheaper than scanning all b data blocks.

Clustering Index – Sparse, on a Non-Key Ordering Field

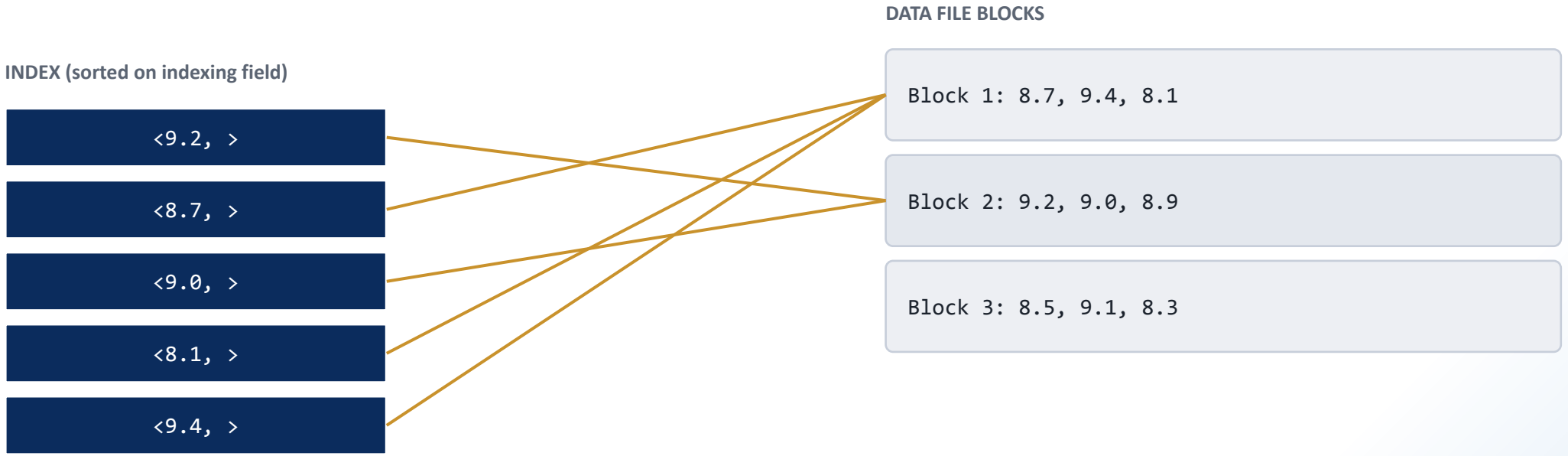
Data file is sorted on a non-key field (e.g., Dept_ID). One index entry per distinct field value, pointing to the first block holding that value.



Key difference from a primary index: the indexing field is not unique, so a value can span multiple blocks (e.g., ECE spans blocks 2 and 3) — the index still points only to the first one.

Secondary Index – Dense, on Any Field

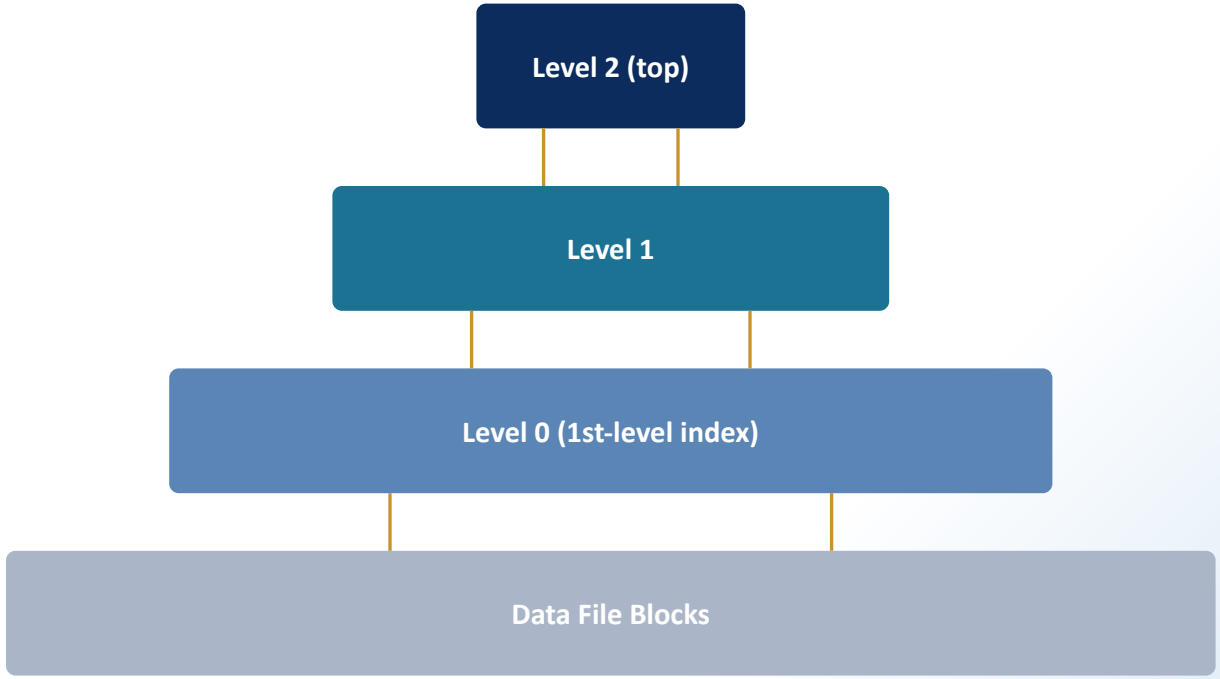
Data file order is unrelated to this field (e.g., CGPA). The index has one entry for every record, so it can locate records even in an unsorted file.



Trade-off: dense indexes are larger (one entry per record) and cost more to maintain on updates, but they support lookups on non-ordering fields and unsorted files.

From a Single Large Index to a Tree of Indexes

- A single-level index is itself a file — as the data file grows, the index outgrows main memory and needs its own block-by-block binary search.
- Idea: treat the first-level index as an ordered file and build a second-level (sparse) index over it — and repeat, until the top level fits in one block.
- This converts an $O(\log_2 b)$ binary search into a fast top-down traversal, touching exactly one block per level.



Search touches exactly one block at each level, top to bottom

Fan-out (fo): number of entries per index block. With fan-out fo, a multilevel index needs only $\lceil \log_{fo}(b_1) \rceil$ levels, where b_1 is the number of first-level index blocks.

Why Static Multi-Level Indexes Fall Short

A static multilevel index (built once, like the one on the previous slide) behaves like the original static single-level index it was meant to fix:

Insertion overflow

A new record can force an entry into an already-full index block, which must then be reorganized or overflowed — index order breaks down.

Deletion gaps

Removing records leaves index blocks sparsely filled, wasting space and adding unnecessary levels over time.

Costly rebuilds

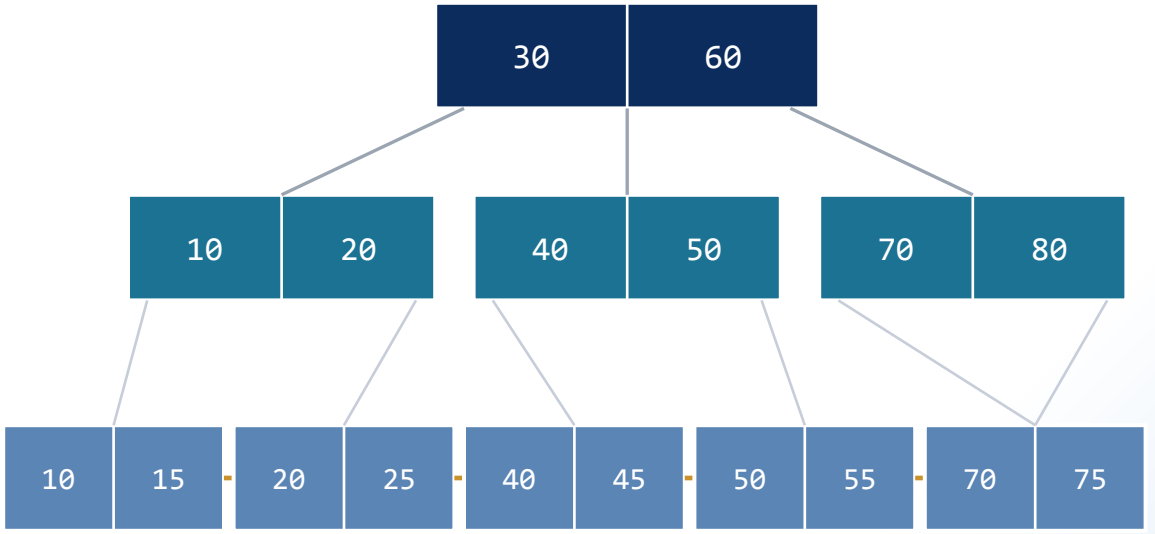
Keeping every level perfectly sorted and full again means periodically reconstructing the entire multilevel structure.

The Fix: Dynamic Multilevel Index

Allow index blocks to be partially full (typically kept between half-full and completely full) and reorganize only locally — splitting or merging nodes — on each insertion or deletion. The B-Tree and B+ Tree are the classic data structures built on this idea; B+ Trees are the ones actually used by virtually every RDBMS index.

B+ Tree – Structure & Properties

Internal nodes hold only search keys and pointers to child nodes — used purely to guide the search.
Leaf nodes hold every search key together with its data pointer (or the actual record), plus a pointer to the next leaf.
Leaf pointer chain links all leaves left-to-right, giving fast ordered (range) scans without revisiting internal nodes.



Leaf chain ⟶ sequential range scans

Order p

An internal node holds up to p – 1 keys and p pointers; a leaf holds up to p – 1 keys. Except for the root, every node must stay at least half-full — ⌈p/2⌉ pointers minimum.

B+ Tree – Search, Insertion & Deletion

Search

- Start at the root; compare the key against separator values.
- Follow the matching child pointer down, level by level.
- At the leaf, scan its entries for the exact key — $O(\log_e N)$ block accesses.

Insertion

- Search to the correct leaf and insert the new key in order.
- If the leaf overflows (> p□ – 1 keys), split it into two leaves.
- Push the split's middle/first key up to the parent; splitting can propagate to the root, which grows the tree by one level.

Deletion

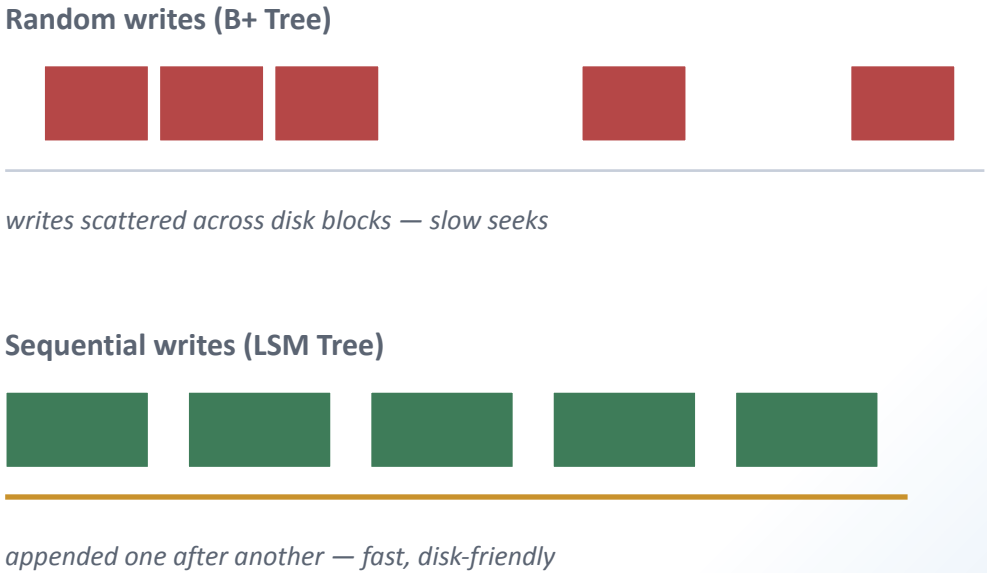
- Search to the leaf and remove the key.
- If the leaf underflows (< ⌈p□/2⌉ keys), borrow an entry from a sibling.
- If borrowing isn't possible, merge with a sibling; merging can propagate upward and shrink the tree.

B+ Tree vs. B-Tree

Aspect	B-Tree	B+ Tree
Data pointers	In internal AND leaf nodes	Only in leaf nodes
Leaf linking	No sequential leaf pointers	Leaves linked — fast range scans
Key duplication	Each key stored once	Separator keys repeated in leaves
Fan-out per node	Lower (space used by data pointers)	Higher (internal nodes store keys only)
Search time for a key	Can end early at an internal node	Always travels to a leaf — predictable
Typical use	Rarely used in practice today	Standard index structure in almost every RDBMS

Log-Structured Merge Trees – Motivation

- B+ Trees update data in place — every write is a random disk I/O to find and modify the right leaf. This is expensive for write-heavy workloads.
- The LSM Tree instead buffers writes in memory and flushes them sequentially to disk, converting random writes into fast sequential writes.
- Reads may need to check multiple on-disk structures, so LSM Trees trade some read performance for much higher write throughput.
- Widely used in systems built for high write volume: Cassandra, HBase, RocksDB, LevelDB, and InfluxDB.



Named for: the “log-structured” append-only write pattern, combined with a “merge” process that periodically consolidates data in the background.

LSM Tree Architecture

1

Write-Ahead Log (WAL)

Every write is first appended to an on-disk log for durability, so a crash before a flush doesn't lose data.

2

MemTable

An in-memory sorted structure (e.g., skip list) receives all writes; reads check it first for the most recent value.

3

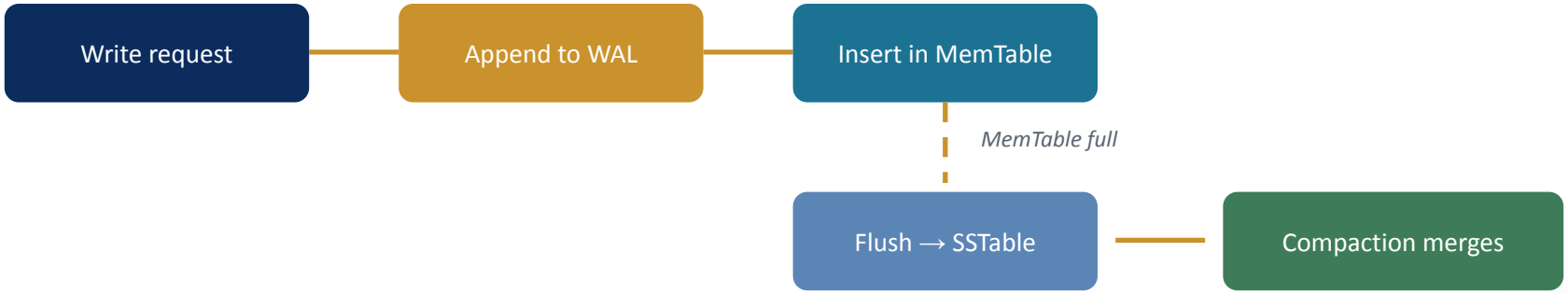
SSTables

When the MemTable is full, it's flushed as an immutable, sorted Sorted String Table on disk.

4

Compaction

A background process merges multiple SSTables, discarding overwritten/deleted keys and controlling their number.



Read path

Check MemTable → newest SSTable → older SSTables, until the key is found (Bloom filters skip SSTables that can't contain it).

LSM Tree Variants – Compaction Strategies

Tiered Compaction

- SSTables of similar size are grouped into tiers; a tier merges into one larger SSTable once it fills up.
- Favors write throughput — fewer, larger merge operations.
- Read cost is higher: a key may need checking across many SSTables in a tier.
- **Used by: Apache Cassandra (Size-Tiered Compaction Strategy).**

Leveled Compaction

- SSTables are organized into levels of increasing size; each level holds non-overlapping key ranges.
- Favors read performance and space efficiency — at most one SSTable per level to check.
- Write amplification is higher: data is rewritten as it moves down levels.
- **Used by: LevelDB, RocksDB (default), Cassandra (LCS option).**

Other notable variants: FloDB and bLSM (write-buffer optimizations), WiscKey (separates keys from large values to cut write amplification), and hybrid strategies used in InfluxDB's Time-Structured Merge (TSM) tree.

COMPARISON

B+ Tree vs. LSM Tree

Aspect	B+ Tree	LSM Tree
Update strategy	In-place update at the leaf	Append-only; old versions cleared by compaction
Write pattern	Random disk I/O	Sequential disk I/O (fast)
Write throughput	Moderate	High — optimized for write-heavy workloads
Read latency	Predictable, single tree traversal	May check MemTable + multiple SSTables
Space overhead	Low — one current copy per record	Higher until compaction reclaims stale versions
Typical systems	MySQL (InnoDB), PostgreSQL, Oracle	Cassandra, HBase, RocksDB, LevelDB

Summary

1. **Single-level indexes** — Primary (sparse, ordering key), clustering (sparse, ordering non-key), secondary (dense, any field).

2. **Multi-level indexes** — A tree of indexes over the first-level index; reduces search to one block access per level.

3. **Dynamic multilevel / B+ Tree** — Nodes split and merge on insert/delete to stay half-full to full; leaves are linked for range scans.

4. **LSM Trees** — Buffer writes in memory (MemTable + WAL), flush sequentially as SSTables, and reconcile via compaction.

5. **Choosing a structure** — B+ Trees suit balanced read/write and range queries; LSM Trees suit write-heavy, high-ingest workloads.

Thank You

BACSE202 - Database Systems

Relational Algebra Operators - Translating SQL Queries into Relational Algebra - Tuple Relational Calculus

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Relational Algebra – Introduction

- Relational algebra is a procedural query language, which takes instances of relations as input and yields instances of relations as output. It uses operators to perform queries.
- An operator can be either unary or binary. They accept relations as their input and yield relations as their output.
- Relational algebra is performed recursively on a relation and intermediate results are also considered relations.

Example Instances

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Unary Relational Algebra Operators

- **Select Operation (σ)**
  - It selects tuples that satisfy the given predicate from a relation.
  - **Notation** : $\sigma_p(r)$
  - Where **σ** stands for selection predicate and **r** stands for relation. *p* is prepositional logic formula which may use connectors like **and, or,** and **not**. These terms may use relational operators like =, ≠, ≥, < , >, ≤.

- **Select Operation (σ)**
  - Examples:

$$\sigma_{subject = "database"}(\text{Books})$$

  - Selects tuples from books where subject is 'database'.

Selection (σ)

- Selects rows that satisfy *selection condition*.
- Result is a relation.

  Schema of result is same as that of the input relation.
- Do we need to do duplicate elimination?

sid	sname	rating	age
28	yuppy	9	35.0
~~31~~	~~lubber~~	~~8~~	~~55.5~~
~~44~~	~~guppy~~	~~5~~	~~35.0~~
58	rusty	10	35.0

$$\sigma_{rating>8}(S2)$$

- **Select Operation (σ)**
  - Examples:

$$\sigma_{\text{subject = "database" and price = "450"}}(\text{Books})$$

  - Selects tuples from books where subject is 'database' and 'price' is 450.

- **Project Operation (∏)**
  - It projects column(s) that satisfy a given predicate.
  - Notation : $\prod_{A1, A2, An} (r)$
    Where A_1, A_2 , A_n are attribute names of relation **r**.
  - Duplicate rows are automatically eliminated, as relation is a set.

Projection

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

S2

sname	rating
yuppy	9
lubber	8
guppy	5
rusty	10

$\pi_{\text{sname,rating}}(S2)$

age
35.0
55.5

$\pi_{\text{Age}}(S2)$

- **Project Operation (∏)**
  - Example

    $$\prod_{\text{subject, author}} (\text{Books})$$

  - Selects and projects columns named as subject and author from the relation Books.

- **Union Operation (∪)**
  - It performs binary union between two given relations and is defined as

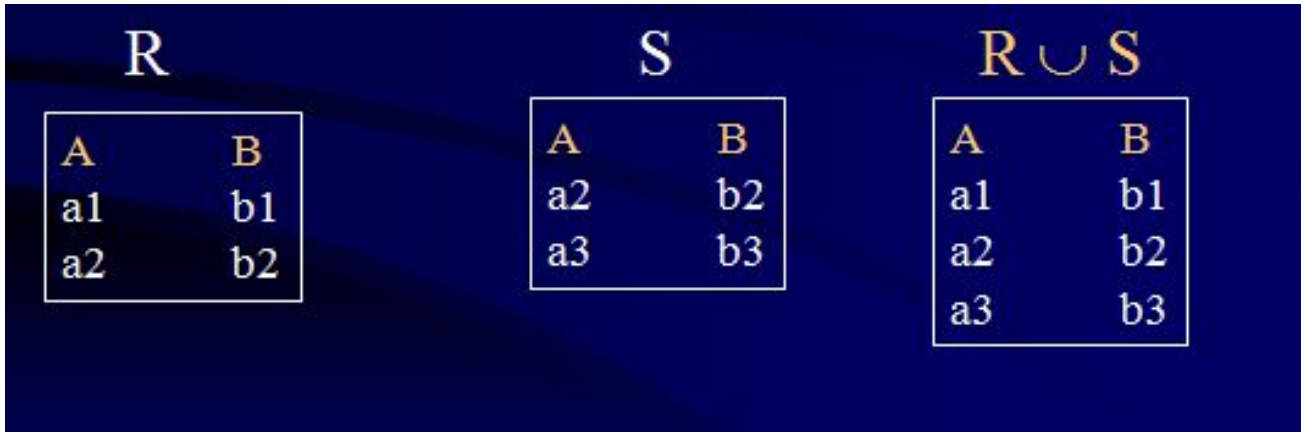
$$r \cup s = \{ t \mid t \in r \text{ or } t \in s \}$$

  - **Notation** : r U s

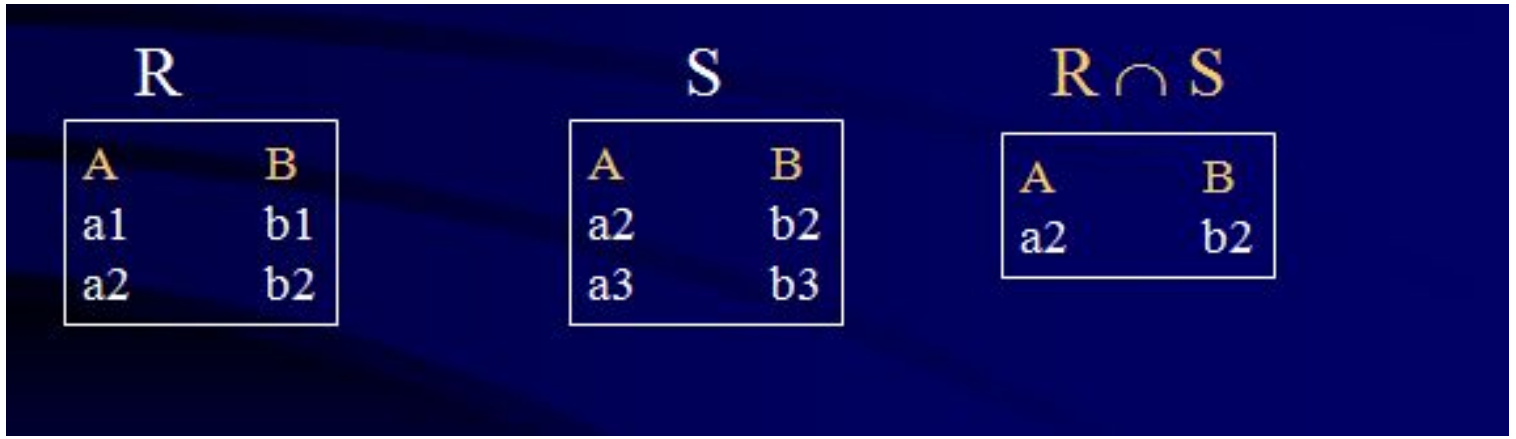
    Where **r** and **s** are either database relations or relation result set (temporary relation).

- **Union Operation (∪)**
  - For a union operation to be valid, the following conditions must hold:
    - **r**, and **s** must have the same number of attributes.
    - Attribute domains must be compatible.
    - Duplicate tuples are automatically eliminated.

- **Union Operation (∪)**
  - Example



- **Intersection Operation**
  - Takes the set of rows that are common to each relation
  - Participating relations must be compatible



Intersection

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S1

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

S2

sid	sname	rating	age
31	lubber	8	55.5
58	rusty	10	35.0

$S1 \cap S2$

- **Set Difference (−)**
  - The result of set difference operation is tuples, which are present in one relation but are not in the second relation.
  - **Notation** : **r − s**
  - Finds all the tuples that are present in **r** but not in **s**.

Set Difference

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S1

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

S2

sid	sname	rating	age
22	dustin	7	45.0

$S1 - S2$

sid	sname	rating	age
28	yuppy	9	35.0
44	guppy	5	35.0

$S2 - S1$

- **Cartesian Product (X)**
  - Combines information of two different relations into one.
  - **Notation** : r X s
  - Where **r** and **s** are relations and their output will be defined as :

$$r \times s = \{ q\ t \mid q \in r \text{ and } t \in s \}$$

JOIN

- SQL Join is used to fetch data from two or more tables, which is joined to appear as single set of data. It is used for combining column from two or more tables by using values common to both tables.

Types of JOIN

- Inner or Equi Join
- Outer
- Left
- Right

Cross JOIN or Cartesian Product

- This type of JOIN returns the cartesian product of rows from the tables in Join. It will return a table which consists of records which combines each row from the first table with each row of the second table.

INNER Join or EQUI Join

- This is a simple JOIN in which the result is based on matched data as per the equality condition specified in the SQL query.
- Inner Join Syntax is,

```
SELECT column-name-list FROM table-name1 INNER JOIN table-name2 WHERE table-name1.column-name = table-name2.column-name;
```

Natural JOIN

- **Natural Join is a type of Inner join which is based on column having same name and same data type present in both the tables to be joined.**
- **The syntax for Natural Join is,**

  SELECT * FROM table-name1 NATURAL JOIN table-name2;

Example of Natural JOIN

Here is the **class** table,

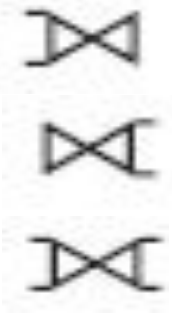
ID	NAME
1	abhi
2	adam
3	alex
4	anu

and the **class_info** table,

ID	Address
1	DELHI
2	MUMBAI
3	CHENNAI

OUTER JOIN

- **Outer Join is based on both matched and unmatched data. Outer Joins subdivide further into,**
  - Left Outer Join
  - Right Outer Join
  - Full Outer Join



LEFT Outer Join

- The left outer join returns a result set table with the matched data from the two tables and then the remaining rows of the left table and null from the right table's columns.
- Syntax for Left Outer Join is,

  SELECT column-name-list FROM table-name1 LEFT OUTER JOIN table-name2 ON table-name1.column-name = table-name2.column-name;

RIGHT Outer Join

- The right outer join returns a result set table with the matched data from the two tables being joined, then the remaining rows of the right table and null for the remaining left table's columns.
- Syntax for Right Outer Join is,

  SELECT column-name-list FROM table-name1 RIGHT OUTER JOIN table-name2 ON table-name1.column-name = table-name2.column-name;

Full Outer Join

- The full outer join returns a resultset table with the matched data of two table then remaining rows of both left table and then the right table.
- Syntax of Full Outer Join is,

` SELECT column-name-list FROM table-name1 FULL OUTER JOIN table-name2 ON table-name1.column-name = table-name2.column-name;

Thank You

BACSE202 - Database Systems

Query Optimization: Query Trees and Heuristic Optimization of Query Trees.

Dr. Somasundaram R

Assistant Professor Sr. Grade 2

Emp Id: 23770

Department of IoT

School of Computer Science and Engineering (SCOPE)

Introduction

- **Query: A query is a request for information from a database.**
- **Query Plans: A query plan (or query execution plan) is an ordered set of steps used to access data in a SQL relational database management system.**
- **Query Optimization: A single query can be executed through different algorithms or re-written in different forms and structures.**
  - Hence, the question of query optimization comes into the picture – Which of these forms or pathways is the most optimal?
  - The query optimizer attempts to determine the most efficient way to execute a given query by considering the possible query plans.

Importance of Query Optimization

The goal of query optimization is to reduce the system resources required to fulfill a query, and ultimately provide the user with the correct result set faster.

- **First, it provides the user with faster results, which makes the application seem faster to the user.**
- **Secondly, it allows the system to service more queries in the same amount of time, because each request takes less time than un-optimized queries.**
- **Thirdly, query optimization ultimately reduces the amount of wear on the hardware (e.g. disk drives), and allows the server to run more efficiently (e.g. lower power consumption, less memory usage).**

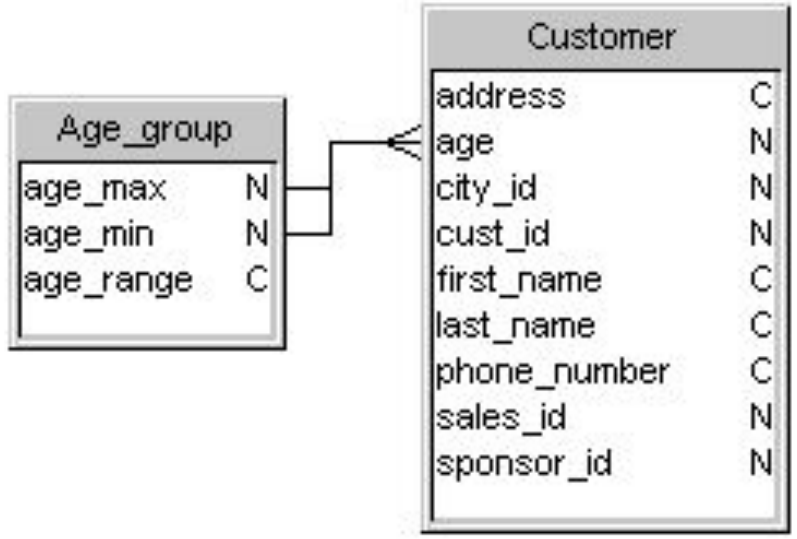
Way to Optimize the Query

- **Analyze and transform equivalent relational expressions: Try to minimize the tuple and column counts of the intermediate and final query processes.**

- **Using different algorithms for each operation: These underlying algorithms determine how tuples are accessed from the data structures they are stored in, indexing, hashing, data retrieval and hence influence the number of disk and block accesses (discussed in query processing).**

Theta Join

- When there is no common column between any two tables, so you cannot use an equi-join. Then you may choose Theta Join to join those two tables if you could find a relationship between them



- **You create a theta join using the operator "Between" for maximum age range and minimum age ranges. By using a theta join, you infer that a join exists where the value in a row of the Age column in the Customer table is between the values in a row for the Age_Min and Age_Max columns of the Age_Group table**

Age Min	Age Max	Age Group
18	30	18-30
31	60	30-60
61	100	Over 60

Age	Customer
18	Arai
64	Baker
19	Brendt
18	Diemers
72	Dupont
36	Dernstein
18	Edwards

Theta Join

Theta join result set

Customer	Age Group
Arai	18-30
Baker	Over 60
Brendt	18-30
Diemers	18-30
Dupont	Over 60
Dernstein	30-60
Edwards	18-30

```
SELECT
  Customer.last_name,
  Age_group.age_range
FROM
  Customer,
  Age_group
WHERE
  ( Customer.age between Age_group.age_min and Age_group.age_max  )
```

Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ldots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.

a) $\sigma_{\theta}(E_1 \, X \, E_2) = E_1 \bowtie_{\theta} E_2$

b) $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

Equivalence Rules

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6. (a) Natural join operations are associative:

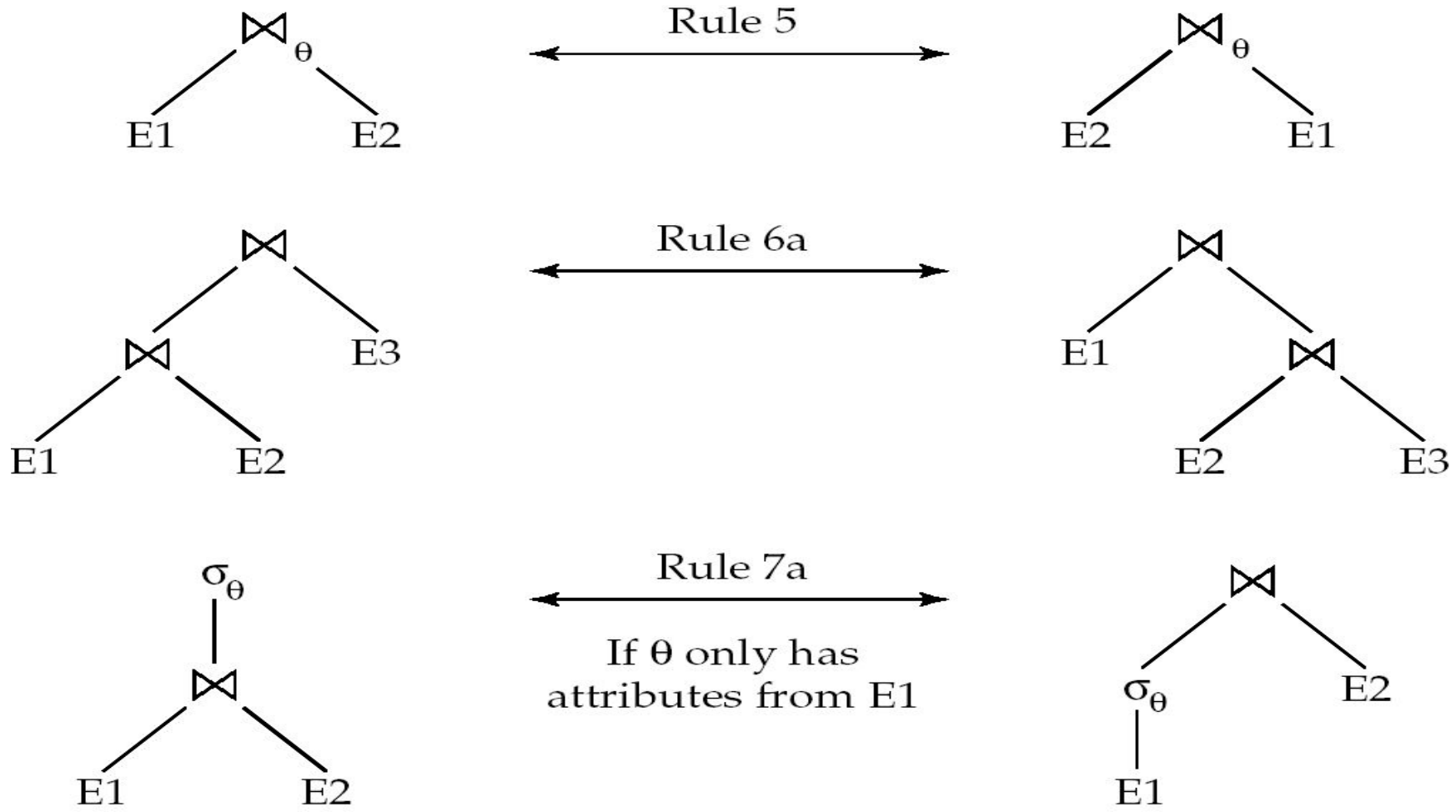
$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3.

Pictorial Depiction of Equivalence Rules



Equivalence Rules

7. The selection operation distributes over the theta join operation under the following two conditions:
   (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta 0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 0}(E_1)) \bowtie_{\theta} E_2$$

   (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta 1 \wedge \theta 2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta 1}(E_1)) \bowtie_{\theta} (\sigma_{\theta 2}(E_2))$$

Equivalence Rules

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$$

(b) Consider a join $E_1 \ _\theta \ E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ, but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ, but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$$

Equivalence Rules

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

- (set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also: $$\sigma_\theta (E_1 - E_2) = \sigma_\theta(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

There are two methods of query optimization

- **Cost based Optimization (Physical)**
- **Heuristic Optimization (Logical)**

Cost based Optimization (Physical)

- It is based on the cost of the query.
- The query can use different paths based on indexes, constraints, sorting methods etc.
- This method mainly uses the statistics like record size, number of records, number of records per block, number of blocks, table size, whether whole table fits in a block, organization of tables, uniqueness of column values, size of columns etc.

- **Cost difference between evaluation plans for a query can be enormous**
- **Steps in cost-based query optimization**
  - Generate logically equivalent expressions using equivalence rules
  - Annotate resultant expressions to get alternative query plans
  - Choose the cheapest plan based on estimated cost
- **Estimation of plan cost based on:**
  - Statistical information about relations. Examples:
    - number of tuples, number of distinct values for an attribute
  - Statistics estimation for intermediate results
    - to compute cost of complex expressions
- **Cost formulae for algorithms, computed using statistics**

- $\sigma_{marks > 60}(\pi_{marks}(student_marks))$
- $\pi_{marks}(\sigma_{marks > 60}(student_marks))$

Processing a Query

- **Tasks in processing a high-level query**
  1. Scanner scans the query and identifies the language tokens
  2. Parser checks syntax of the query
  3. The query is validated by checking that all attribute names and relation names are valid
  4. An intermediate internal representation for the query is created (query tree or query graph)
  5. Query execution strategy is developed
  6. Query optimizer produces an execution plan
  7. Code generator generates the object code
  8. Runtime database processor executes the code
- **Query processing and query optimization**

Processing a Query

- **Typical steps in processing a high-level query**
  1. *Query in a high-level query language like SQL*
  2. **Scanning, parsing, and validation**
  3. *Intermediate-form of query like query tree*
  4. **Query optimizer**
  5. *Execution plan*
  6. **Query code generator**
  7. *Object-code for the query*
  8. **Run-time database processor**
  9. *Results of query*

Processing a Query

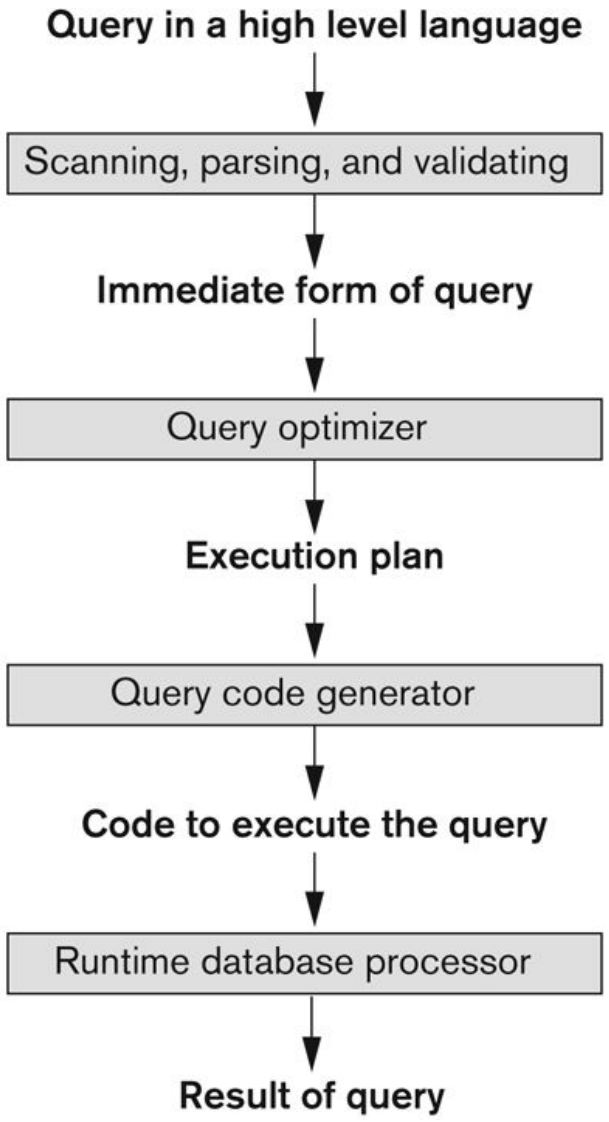
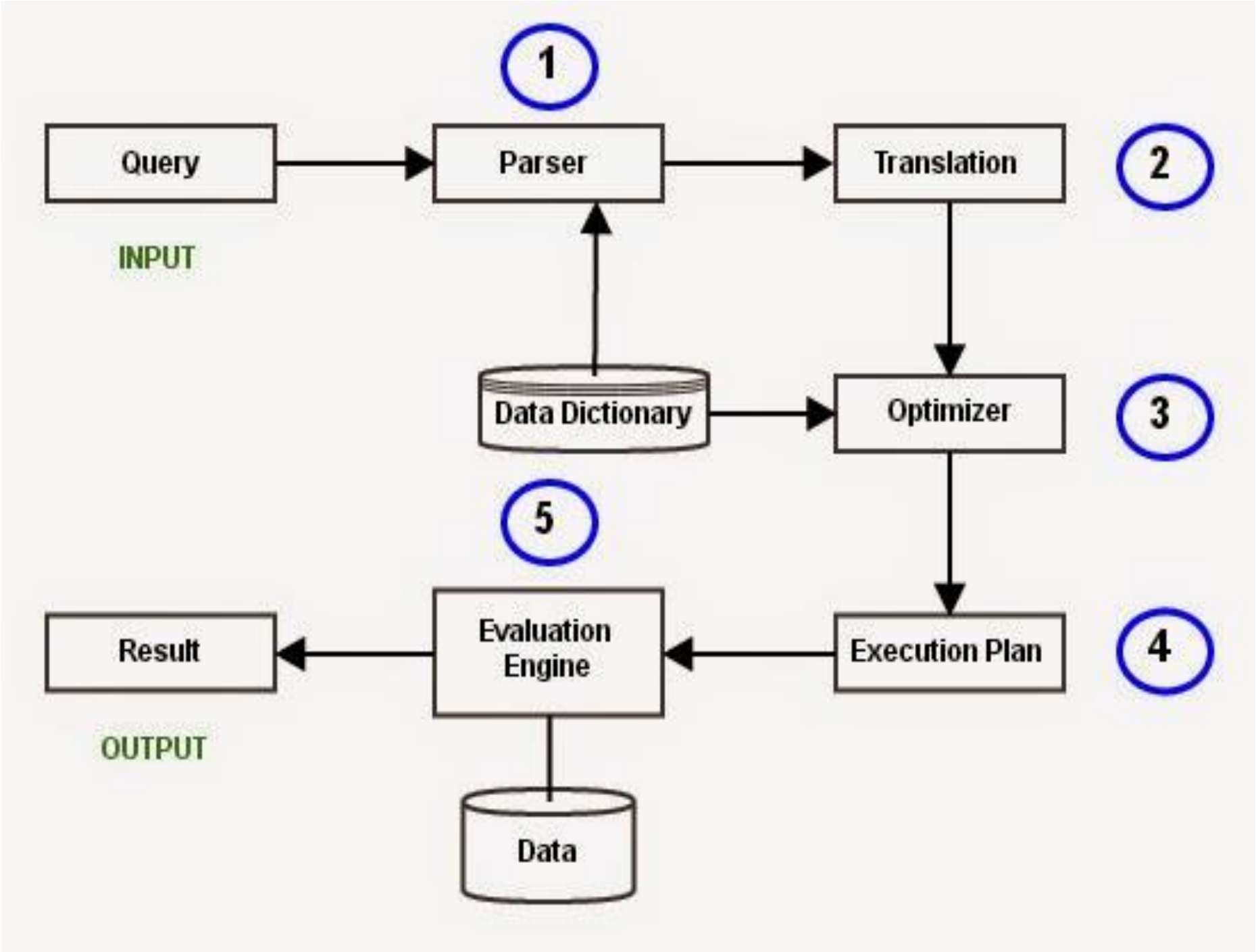


Figure 15.1
Typical steps when processing a high-level query.

Code can be:

Executed directly (interpreted mode)

Stored and executed later whenever needed (compiled mode)


1
Query
Parser
Translation
2
INPUT
Data Dictionary
Optimizer
3
5
Result
Evaluation Engine
Execution Plan
4
OUTPUT
Data

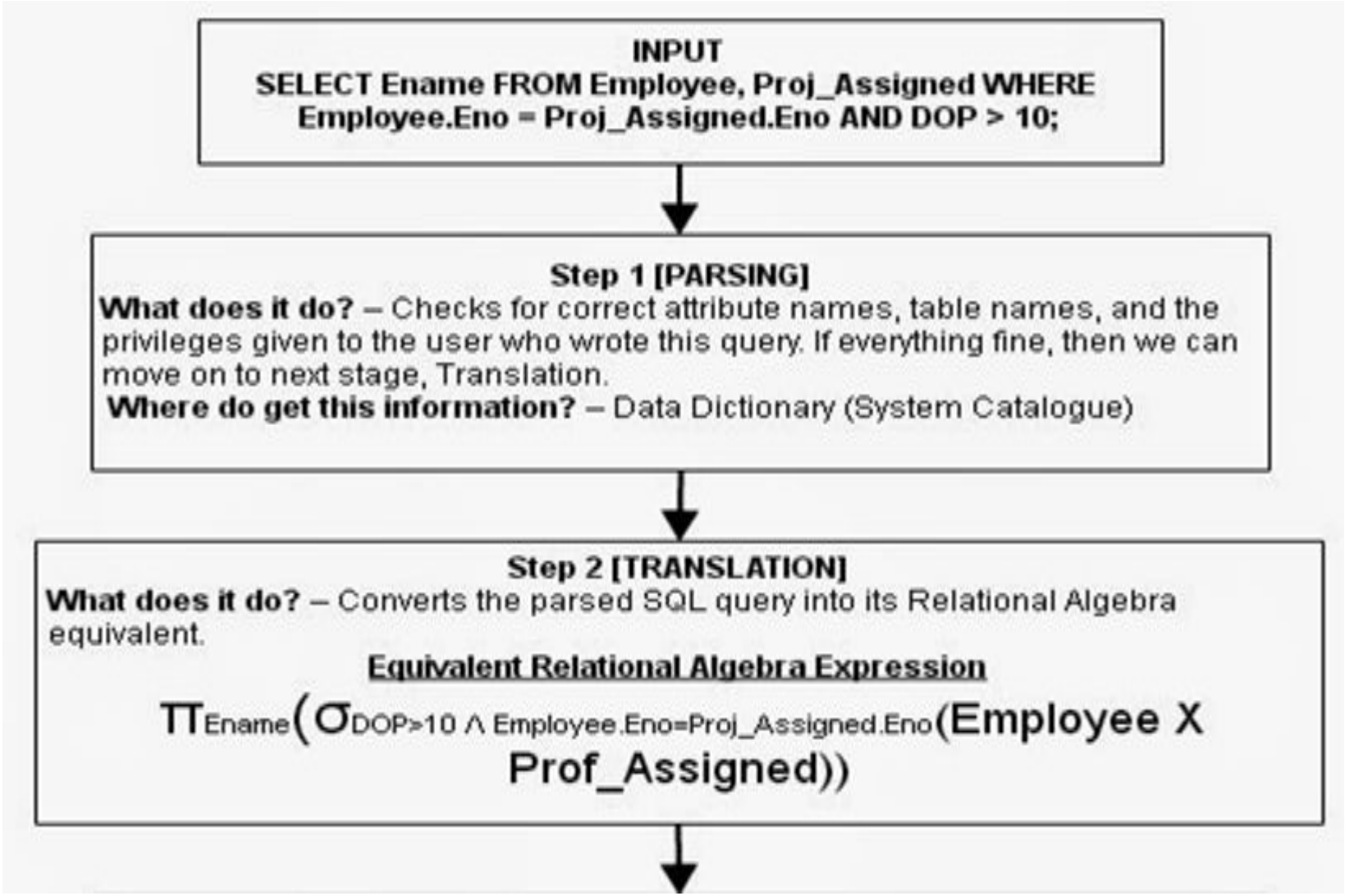
- **Input:**
  - A query written in SQL is given as input to the query processor.
- **Parsing**
  - In this step, the parser of the query processor module checks the syntax of the query, the user's privileges to execute the query, the table names and attribute names, etc. The correct table names, attribute names and the privilege of the users can be taken from the system catalog (data dictionary).

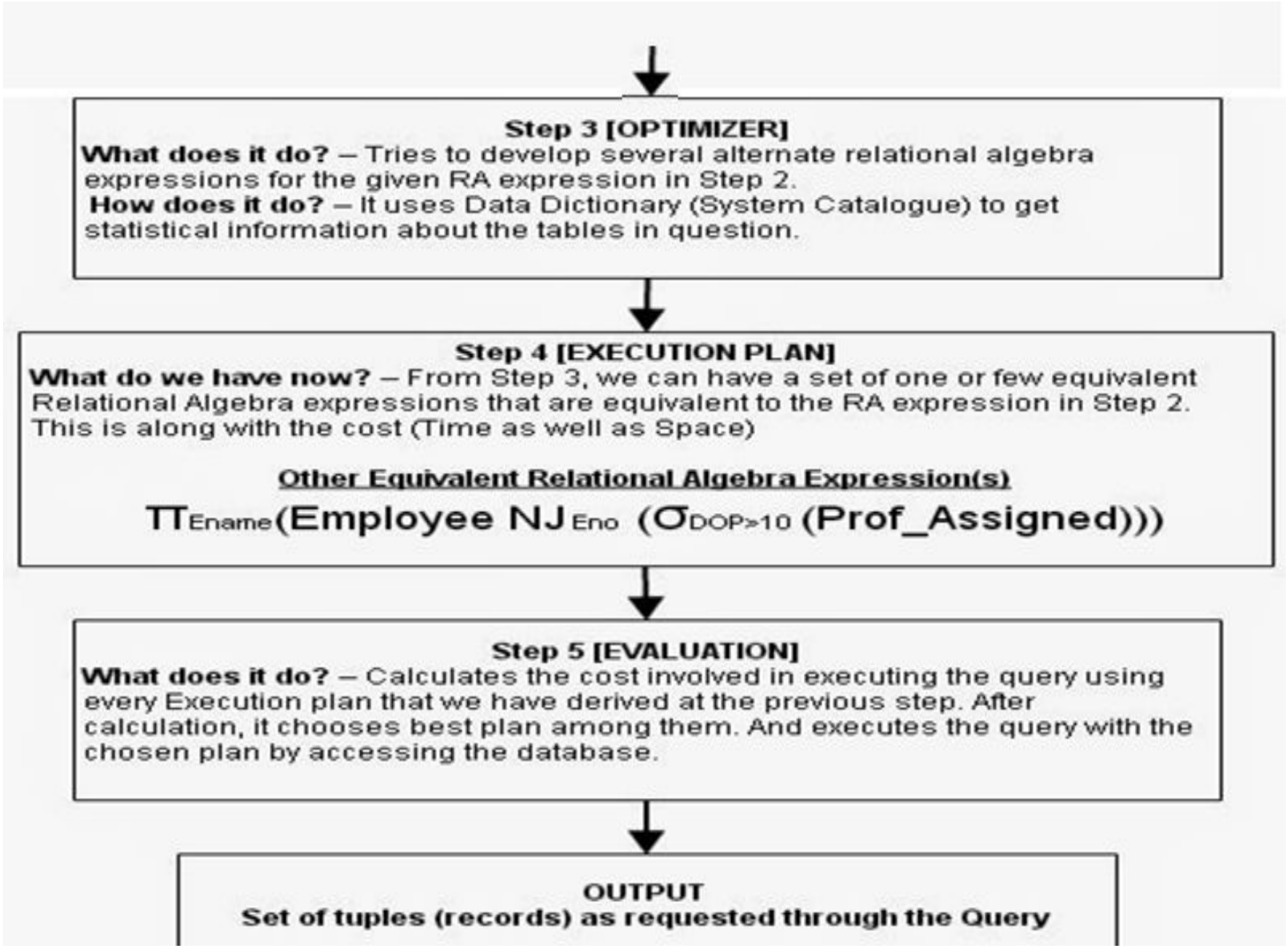
- **Translation**
  - If we have written a valid query, then it is converted from high level language SQL to low level instruction in Relational Algebra.
  - For example, our SQL query can be converted into a Relational Algebra equivalent as follows;
  - SELECT Ename FROM Employee, Proj_Assigned WHERE Employee.Eno = Proj_Assigned.Eno AND DOP > 10;
  - $\pi_{Ename}(\sigma_{DOP>10 \wedge Employee.Eno=Proj_Assigned.Eno}$ (Employee X Prof_Assigned))

- **Optimizer**
  - Optimizer uses the statistical data stored as part of data dictionary. The statistical data are information about the size of the table, the length of records, the indexes created on the table, etc. Optimizer also checks for the conditions and conditional attributes which are parts of the query.

- **Execution Plan**
  - A query can be expressed in many ways. The query processor module, at this stage, using the information collected in step 3 to find different relational algebra expressions that are equivalent and return the result of the one which we have written already.
  - For our example, the query written in Relational algebra can also be written as the one given below;
  - $\pi_{Ename}(Employee \bowtie_{Eno} (\sigma_{DOP>10}(Prof_Assigned)))$
  - So far, we have got two execution plans. Only condition is that both plans should give the same result.

- **Evaluation**
  - At this stage, we choose one execution plan of the several we have developed. This Execution plan accesses data from the database to give the final result.
  - In our example, the second plan may be good. In the first plan, we join two relations (costly operation) then apply the condition (conditions are considered as filters) on the joined relation. This consumes more time as well as space.
  - In the second plan, we filter one of the tables (Proj_Assigned) and the result is joined with the Employee table. This join may need to compare less number of records. Hence, the second plan is the best (with the information known, not always).


INPUT
SELECT Ename FROM Employee, Proj_Assigned WHERE Employee.Eno = Proj_Assigned.Eno AND DOP > 10;
Step 1 [PARSING]
What does it do? – Checks for correct attribute names, table names, and the privileges given to the user who wrote this query. If everything fine, then we can move on to next stage, Translation.
Where do get this information? – Data Dictionary (System Catalogue)
Step 2 [TRANSLATION]
What does it do? – Converts the parsed SQL query into its Relational Algebra equivalent.
Equivalent Relational Algebra Expression
ΠEname(σDOP>10 ∧ Employee.Eno=Proj_Assigned.Eno(Employee X Prof_Assigned))


Step 3 [OPTIMIZER]
What does it do? – Tries to develop several alternate relational algebra expressions for the given RA expression in Step 2.
How does it do? – It uses Data Dictionary (System Catalogue) to get statistical information about the tables in question.
Step 4 [EXECUTION PLAN]
What do we have now? – From Step 3, we can have a set of one or few equivalent Relational Algebra expressions that are equivalent to the RA expression in Step 2. This is along with the cost (Time as well as Space)
Other Equivalent Relational Algebra Expression(s)
TTEname(Employee NJ Eno (σDOP>10 (Prof_Assigned)))
Step 5 [EVALUATION]
What does it do? – Calculates the cost involved in executing the query using every Execution plan that we have derived at the previous step. After calculation, it chooses best plan among them. And executes the query with the chosen plan by accessing the database.
OUTPUT
Set of tuples (records) as requested through the Query

SQL Queries and Relational Algebra (1)

- Example

```
SELECT     Lname, Fname
FROM EMPLOYEE
WHERE Salary > ( SELECT MAX(Salary)
                 FROM EMPLOYEE
                 WHERE Dno = 5  )
```

- Inner block and outer block

Translating SQL Queries into Relational Algebra

```
SELECT    LNAME, FNAME
FROM          EMPLOYEE
WHERE SALARY    >         ( SELECT    MAX (SALARY)
                          FROM          EMPLOYEE
                          WHERE    DNO = 5);
```

```
SELECT    LNAME, FNAME
FROM      EMPLOYEE
WHERE     SALARY > C
```

```
SELECT    MAX (SALARY)
FROM      EMPLOYEE
WHERE     DNO = 5
```

$$\pi_{\text{LNAME, FNAME}} (\sigma_{\text{SALARY>C}}(\text{EMPLOYEE}))$$

$$\mathcal{F}_{\text{MAX SALARY}} (\sigma_{\text{DNO=5}} (\text{EMPLOYEE}))$$

SQL Queries and Relational Algebra (2)

- **Uncorrelated nested queries Vs Correlated nested queries**
- **Example**

Retrieve the name of each employee who works on all the projects controlled by department number 5.

```
SELECT FNAME, LNAME FROM EMPLOYEE
WHERE  ((SELECT PNO FROMWORKS_ON
WHERE SSN=ESSN) CONTAINS      (SELECT      PNUMBER FROM PROJECT
WHERE DNUM=5) )
```

SQL Queries and Relational Algebra (3)

- **Example**

  For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate.

- **SQL query:**

```
SELECT    P.NUMBER,P.DNUM,E.LNAME, E.ADDRESS, E.BDATE
FROMPROJECT AS P,DEPARTMENT AS D, EMPLOYEE AS E
WHERE     P.DNUM=D.DNUMBER AND D.MGRSSN=E.SSN AND
P.PLOCATION='STAFFORD';
```

- **Relation algebra:**

$$\pi_{\text{PNUMBER, DNUM, LNAME, ADDRESS, BDATE}}(((\sigma_{\text{PLOCATION='STAFFORD'}}(\text{PROJECT}))_{\text{DNUM=DNUMBER}}(\text{DEPARTMENT}))_{\text{MGRSSN=SSN}}(\text{EMPLOYEE}))$$

SQL Queries and Relational Algebra (4)

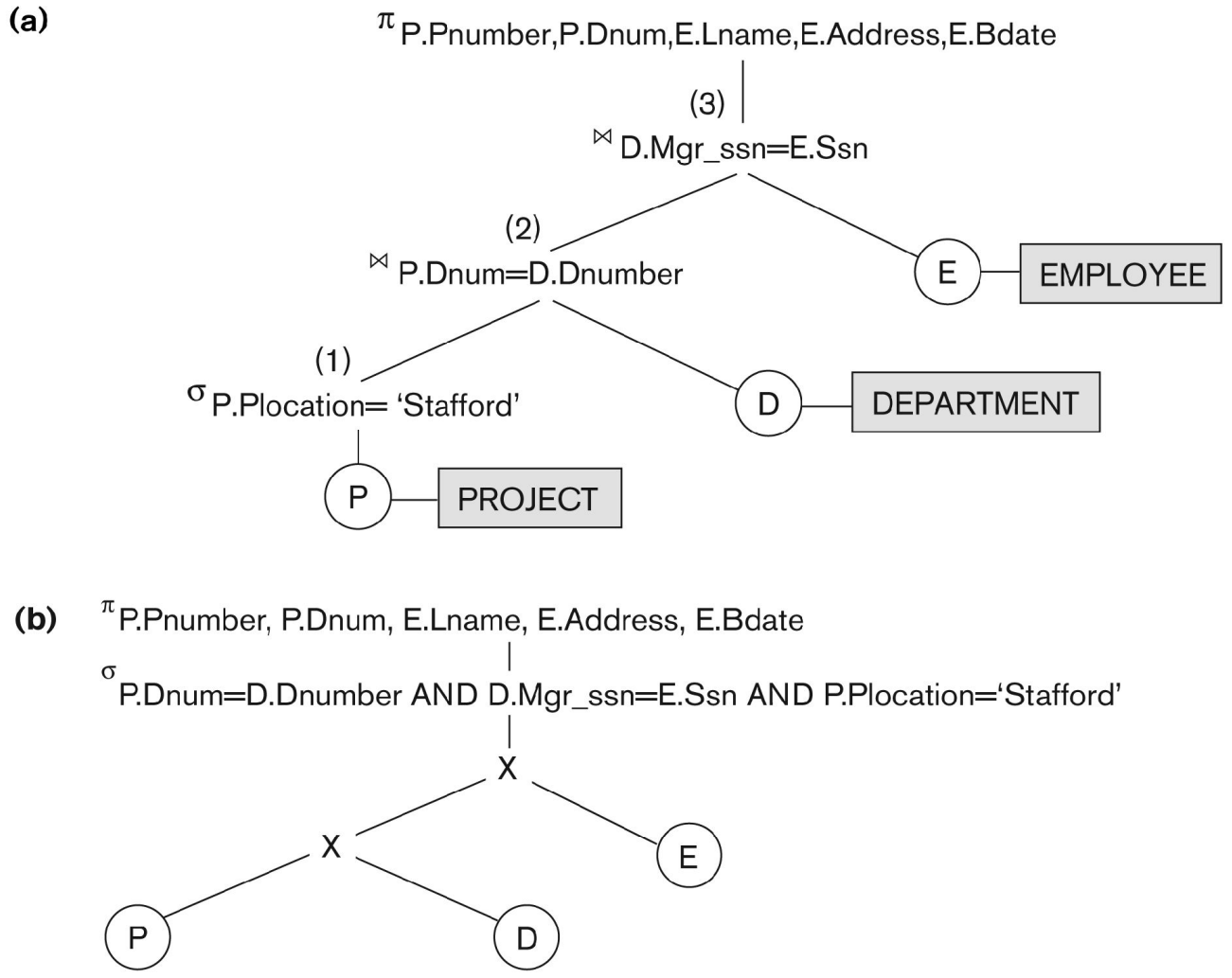


Figure 15.4
Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Heuristic Optimization of Query Trees

Heuristic: Rule that leads to the least cost in most cases

Query Tree (relational algebra expression)

leaf node :relations

Internal node :relational algebra operations

execution of query trees: post order traversal of tree

Lets Take an Example

- Consider the two tables with the data given below
- Empo

```
SQL> SELECT * FROM EMP1;

ID   NAM DEP DES
---  --- --- ---
123  SAM 111 AKJ
124  TOM 222 CMU
125  RAM 333 KCM
126  SAM 222 CMU
127  SAM 333 KCM
```

we have 5 records. Each record occupies 12 bytes
totally we need 60 bytes
to store these record

Dep Table

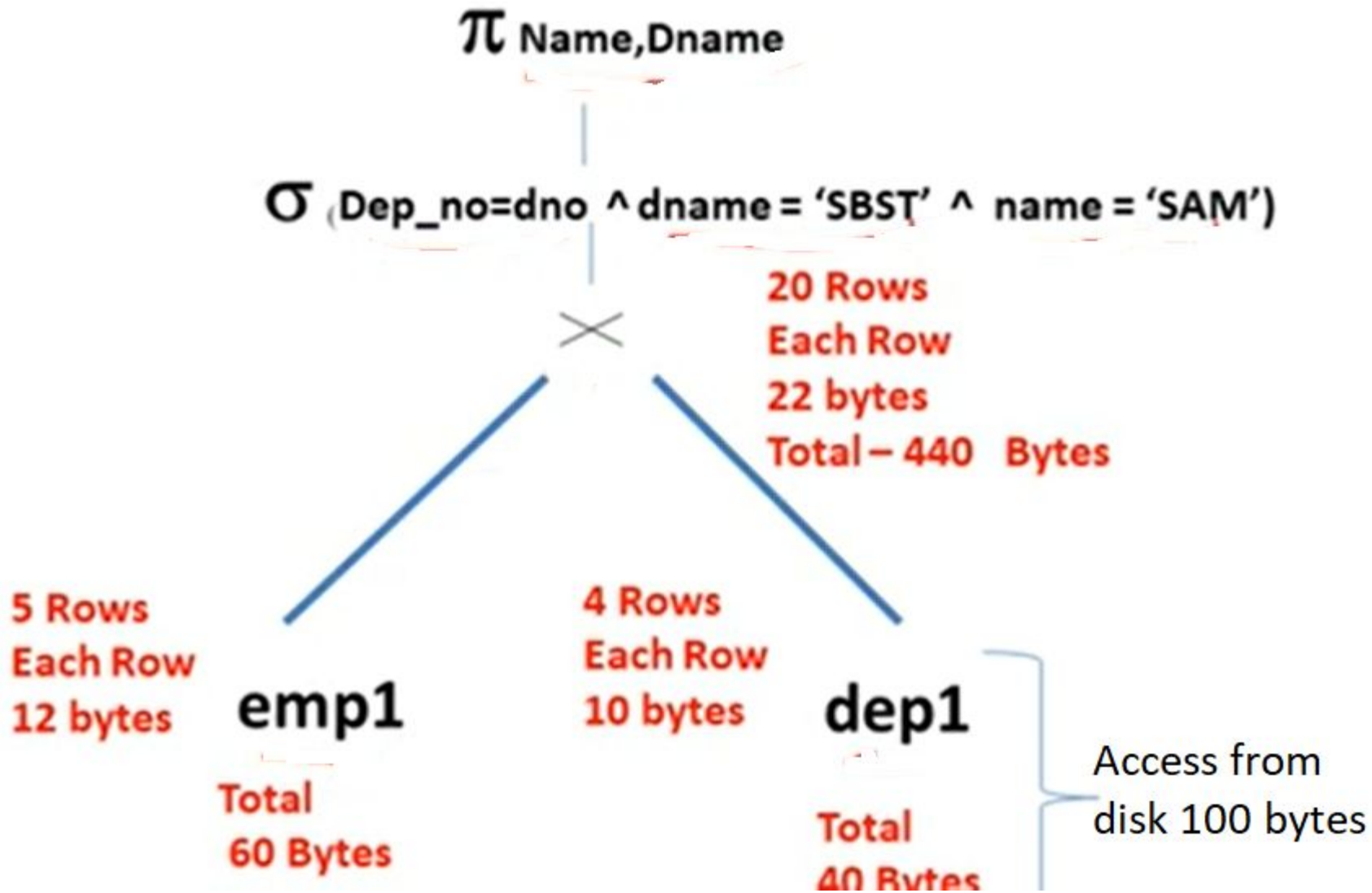
```
SQL> select * from dep1;

DNO  DNAM  DLO
---  ----  ---
111  SBST  SJT
222  SITE  MBD
333  SMEC  SMV
444  SBST  MBD
```

we have 4 records
Each record occupies
10 bytes
so, totally 40 bytes
is needed to store

```
Select name,dname from employee,department
where dep no=dno and name='SAM' and
dname='SBST';
```

Initial Query Tree



emp1 cross Join dep1

```
SQL> select * from emp1 cross join dep1;
```

TEMP1

ID	NAM	DEP	DES	DNO	DNAM	DLO
123	SAM	111	AKJ	111	SBST	sjt
124	TOM	222	CMU	111	SBST	sjt
125	RAM	333	KCM	111	SBST	sjt
126	SAM	222	CMU	111	SBST	sjt
127	SAM	333	KCM	111	SBST	sjt
123	SAM	111	AKJ	222	SITE	MB
124	TOM	222	CMU	222	SITE	MB
125	RAM	333	KCM	222	SITE	MB
126	SAM	222	CMU	222	SITE	MB
127	SAM	333	KCM	222	SITE	MB
123	SAM	111	AKJ	333	SMEC	SMV

ID	NAM	DEP	DES	DNO	DNAM	DLO
124	TOM	222	CMU	333	SMEC	SMV
125	RAM	333	KCM	333	SMEC	SMV
126	SAM	222	CMU	333	SMEC	SMV
127	SAM	333	KCM	333	SMEC	SMV
123	SAM	111	AKJ	444	SBST	MB
124	TOM	222	CMU	444	SBST	MB
125	RAM	333	KCM	444	SBST	MB
126	SAM	222	CMU	444	SBST	MB

TEMP 1 Size –
20 Rows with each 22 bytes long.
Total Bytes – 440

$\sigma_{(Dep_no=dno \wedge dname='SBST' \wedge name='SAM')}(Temp1)$

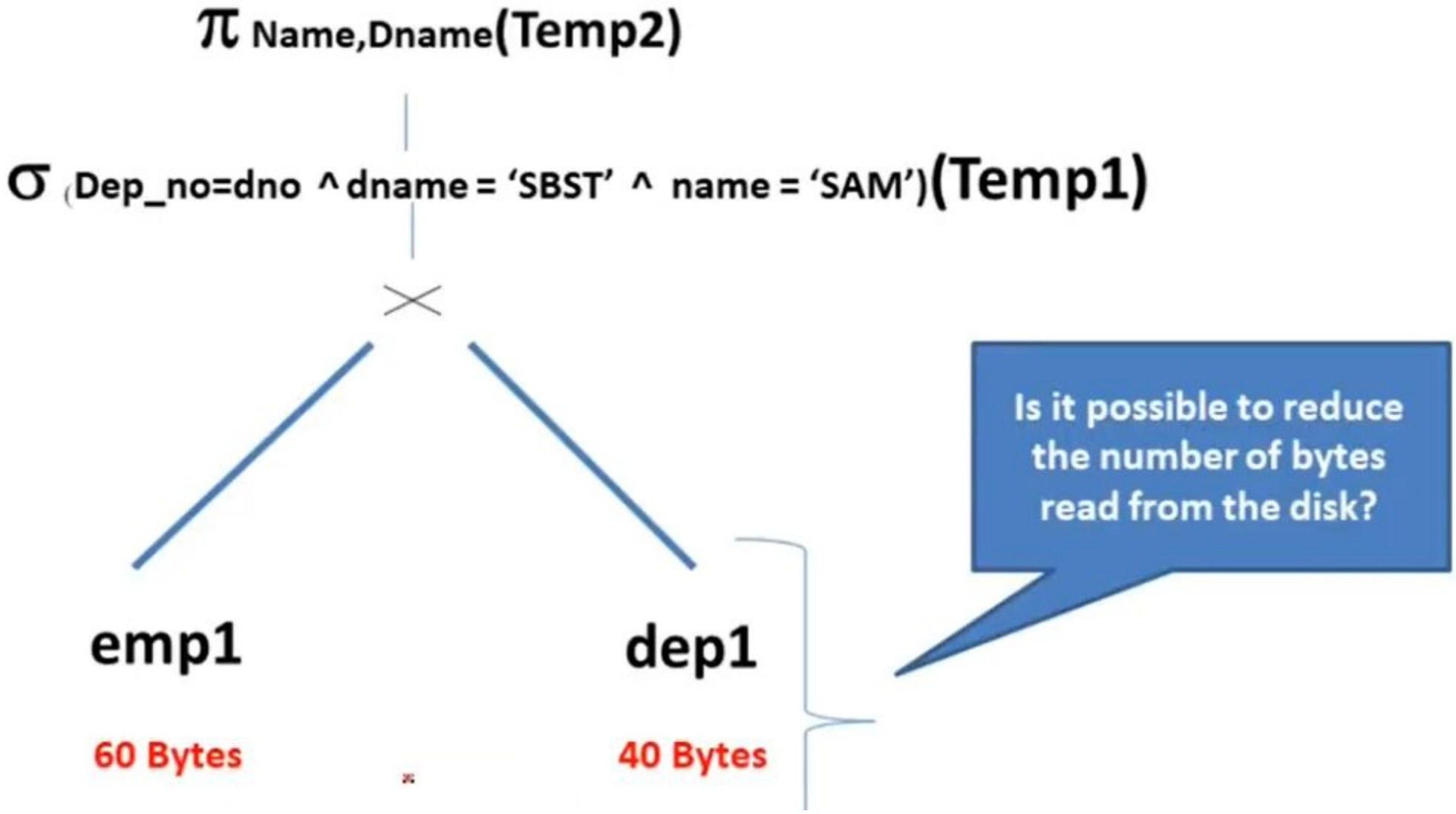
π Name,Dname(Temp2)

ID	NAM	DEP	DES	DNO	DNAM	DLO
123	SAM	111	AKJ	111	SBST	SJT

Final Result

NAM	DNAM
SAM	SBST

Initial Query Tree



RULE 1

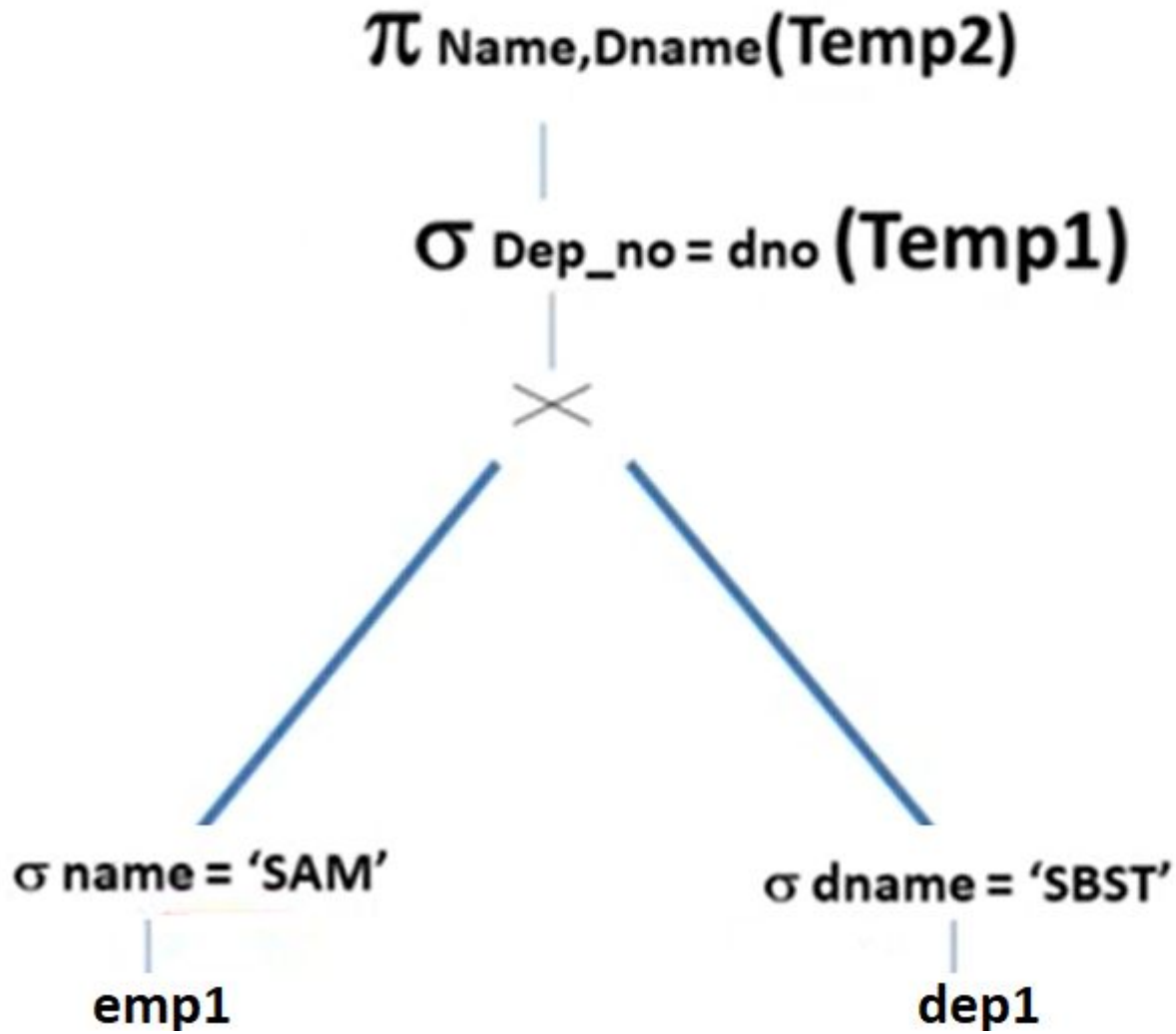
- Cascade of σ
- A conjunctive selection condition can be broken up in to a cascade of individual σ operations

$$\sigma_{(Dep_no=dno \wedge dname = 'SBST' \wedge name = 'SAM')}(Temp1)$$



$$\sigma_{Dep_no=dno}(\sigma_{dname = 'SBST'}(\sigma_{name = 'SAM'}(Temp1)))$$

Rule 2 :Move the Select σ **as far as down the tree**



Only Employees with Name **'SAM'**

```
SQL> SELECT * FROM EMP1;

ID   NAM DEP DES
---  --- --- ---
123  SAM 111 AKJ
124  TOM 222 CMU
125  RAM 333 KCM
126  SAM 222 CMU
127  SAM 333 KCM
```

ONLY 3 Records
satisfy the condition
so we need 3 x 12 =36 Bytes

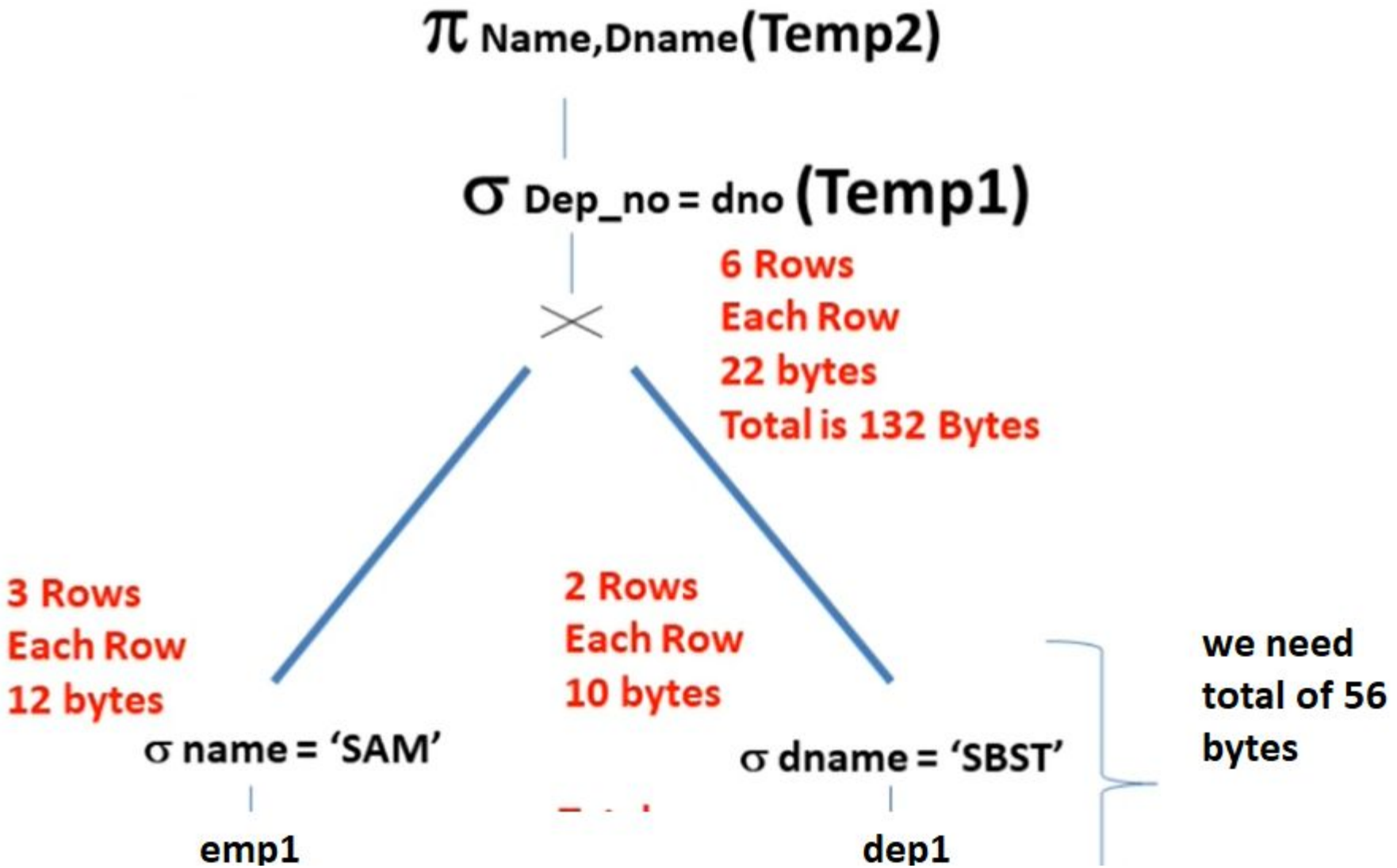
Only Departments with Name 'SBST'

```
SQL> select * from dep1;

DNO  DNAM  DLO
---  ----  ---
111  SBST  SJT
222  SITE  MBD
333  SMEC  SMV
444  SBST  MBD
```

Only 2 records satisfy the condition, so we need 2 x 10= 20 bytes

By Rule 2 We have reduced to a total of 132 Bytes



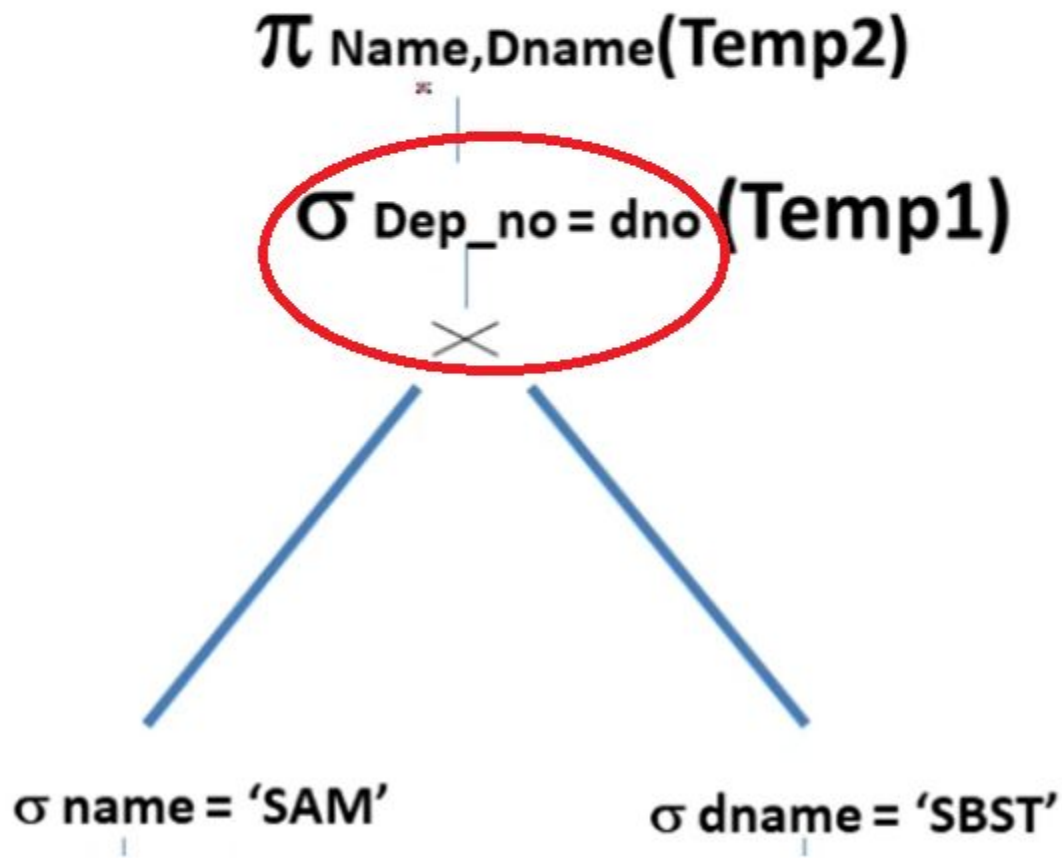
Temp1

ID	NAM	DEP	DES	DNO	DNAM	DLO
123	SAM	111	AKJ	111	SBST	SJT
126	SAM	222	CMU	111	SBST	SJT
127	SAM	333	KCM	111	SBST	SJT
123	SAM	111	AKJ	444	SBST	MBD
126	SAM	222	CMU	444	SBST	MBD
127	SAM	333	KCM	444	SBST	MBD

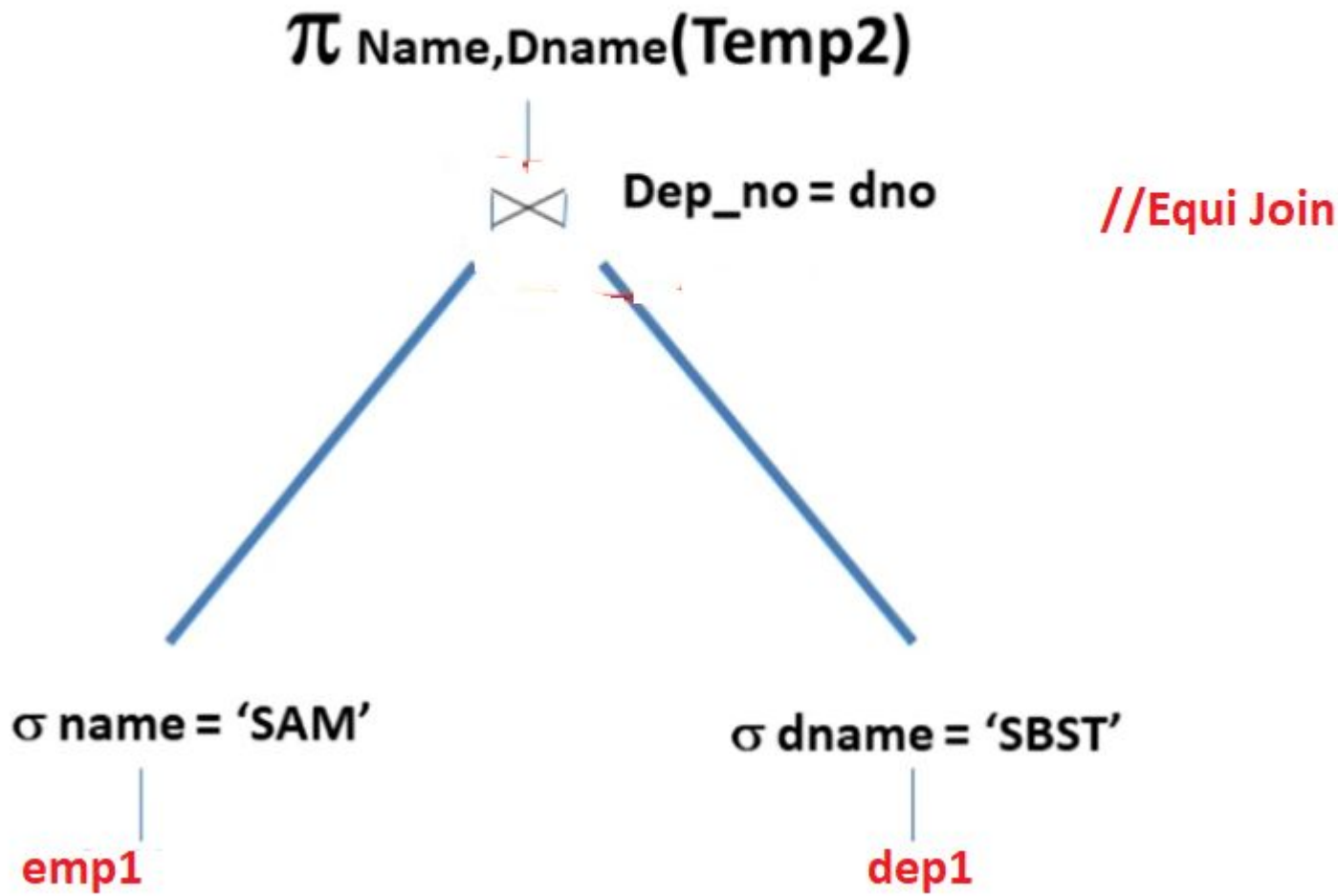
Temp1 is reduced to 132 bytes from 440 Bytes.

RULE 3

- **Combine the Cartesian Product with a subsequent select operation to Join Operation.**



Rule 3 : Combine the Cartesian Product with a subsequent select operation to a Join Operation



When Converting to Equi join, Instead of 6 rows we need only one row

EMP1

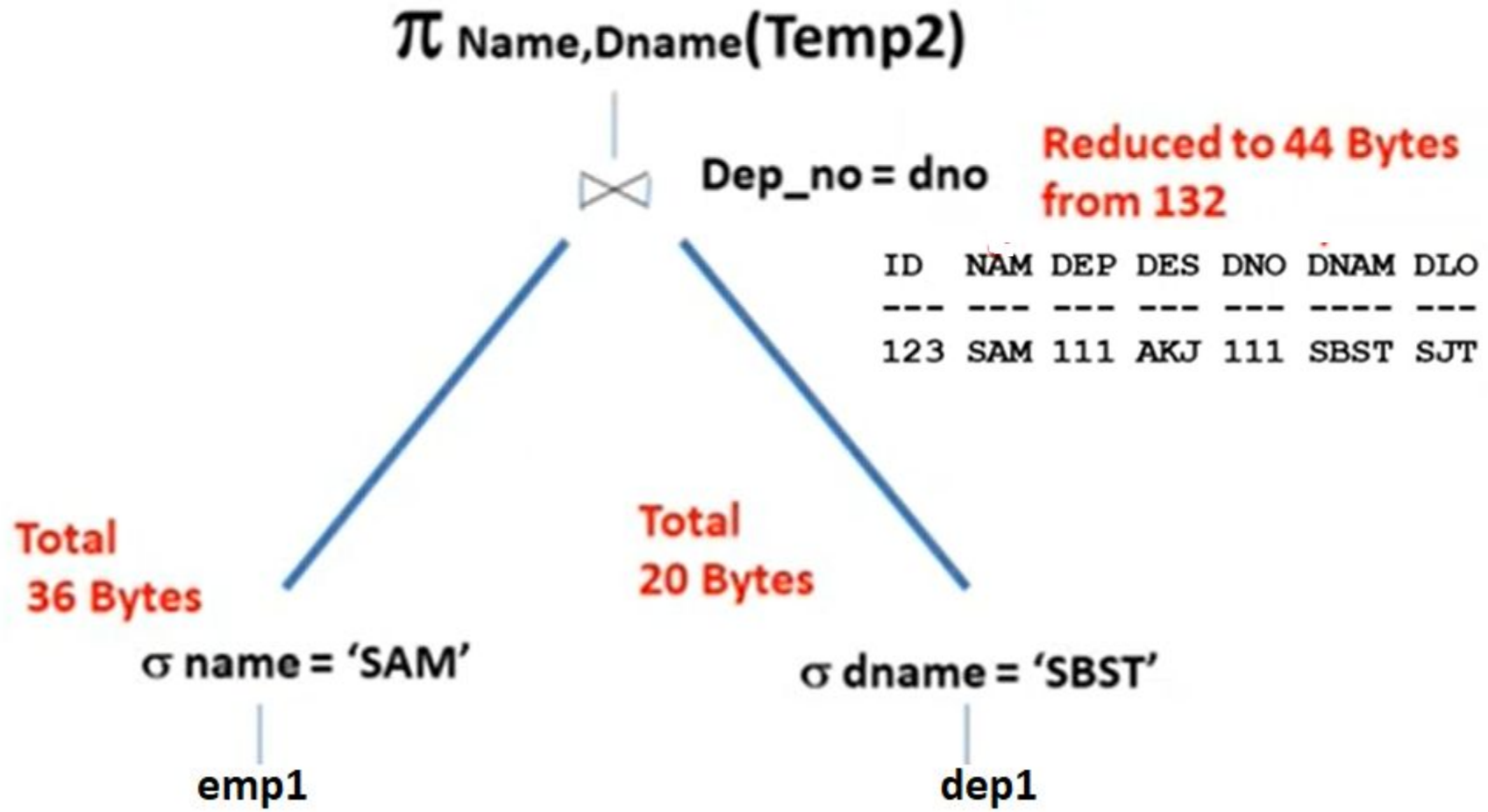
ID	NAM	DEP	DES
123	SAM	111	AKJ
126	SAM	222	CMU
127	SAM	333	KCM



DEP1

DNO	DNAM	DLO
111	SBST	SJT
444	SBST	MBD

Rule 3 : Combine the Cartesian Product with a subsequent select operation to a Join Operation



Rule 4

- Break Down the Project operations down the tree as far as possible.

```
SQL> SELECT * FROM EMP1;

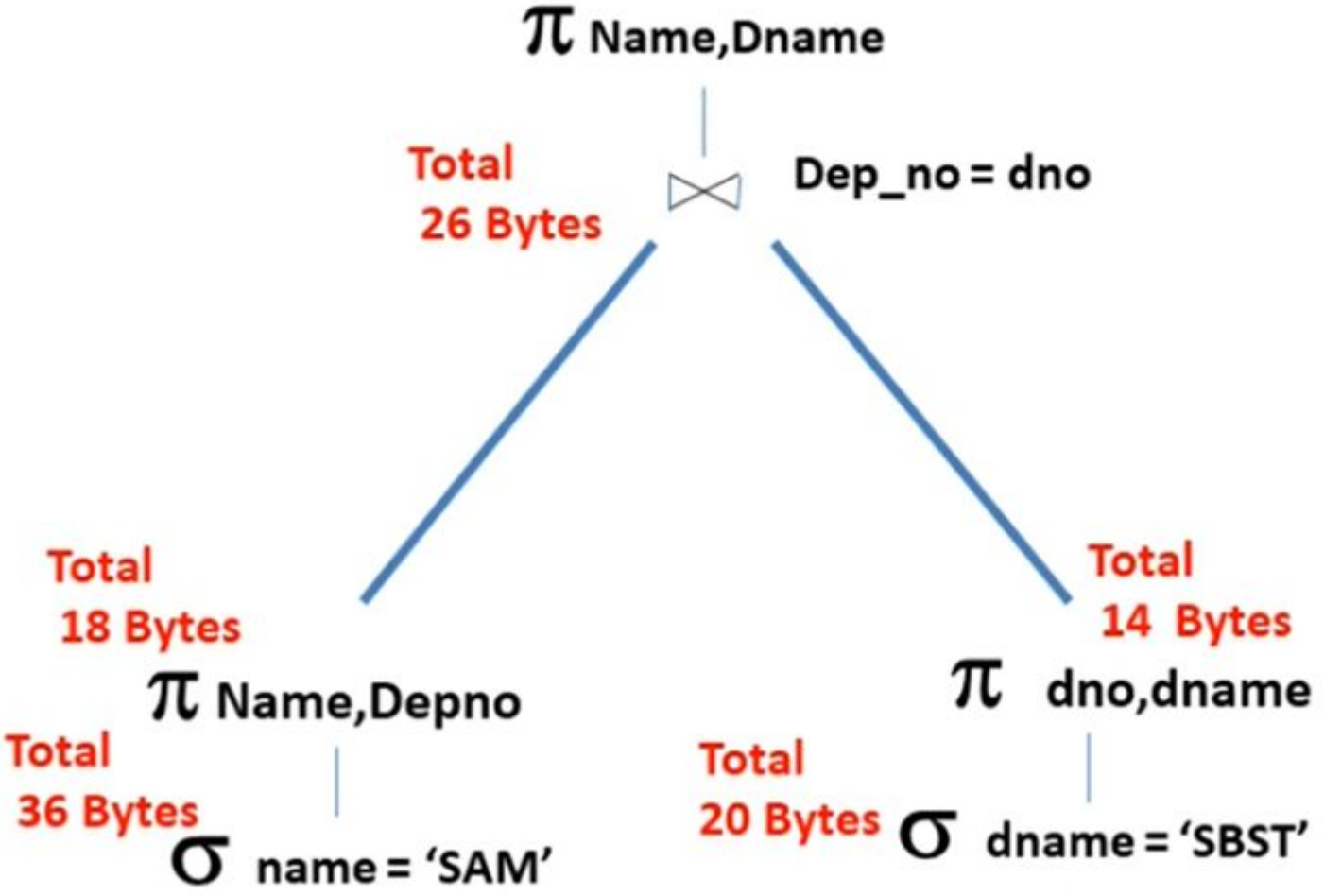
ID   NAM DEP DES
---  --- --- ---
123  SAM 111 AKJ
124  TOM 222 CMU
125  RAM 333 KCM
126  SAM 222 CMU
127  SAM 333 KCM
```

DEP1 Table

DNO	DNAM	DLO
111	SBST	SJT
222	SITE	MBD
333	SMEC	SMV
444	SBST	MBD

Break the Project Operations and Move it down the tree

Rule 4 : Break and Move Project Operations down the tree as far as possible



NAM	DEP
SAM	111
SAM	222
SAM	333

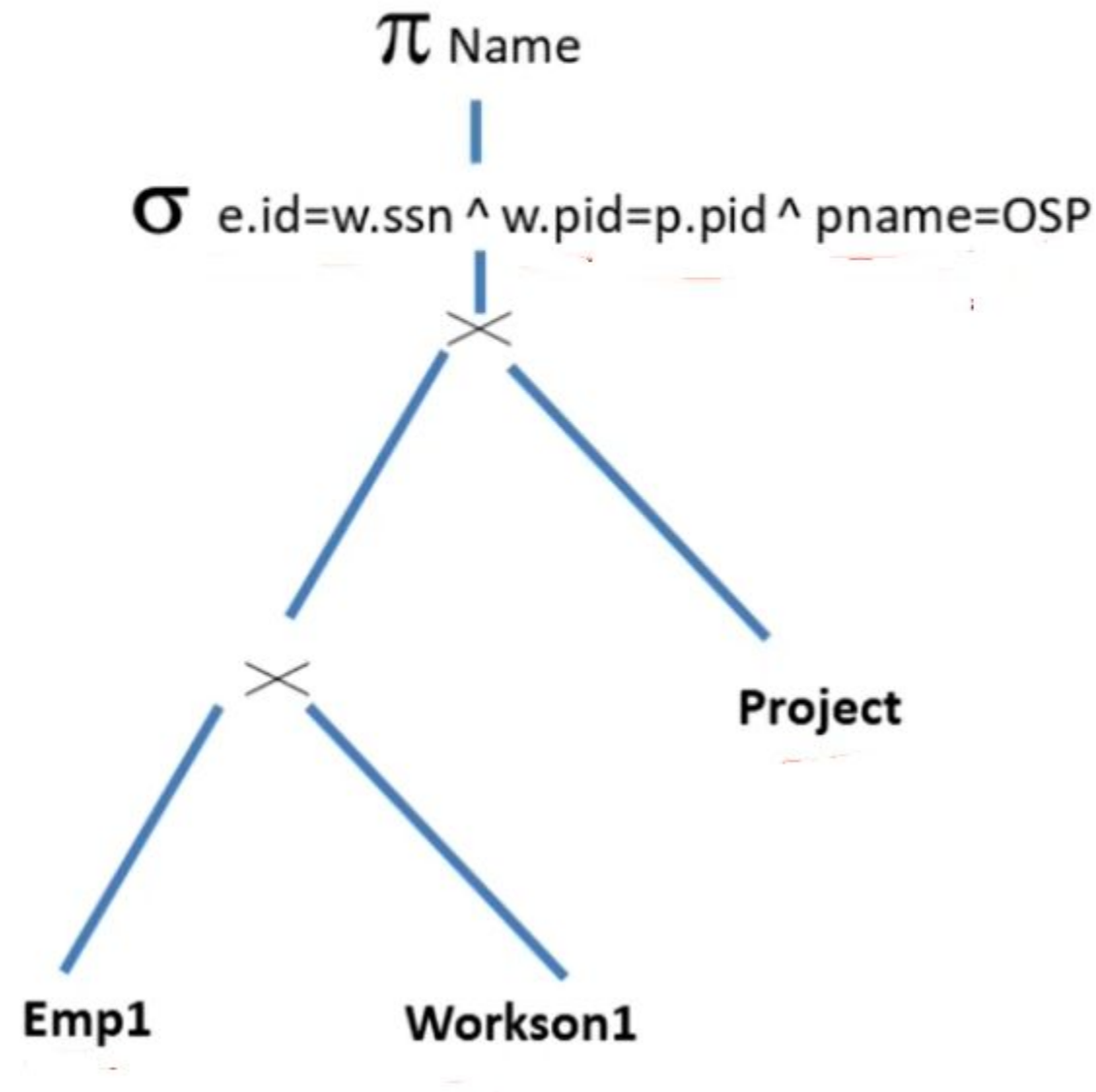
DNO	DNAM
111	SBST
444	SBST

emp1
dep1

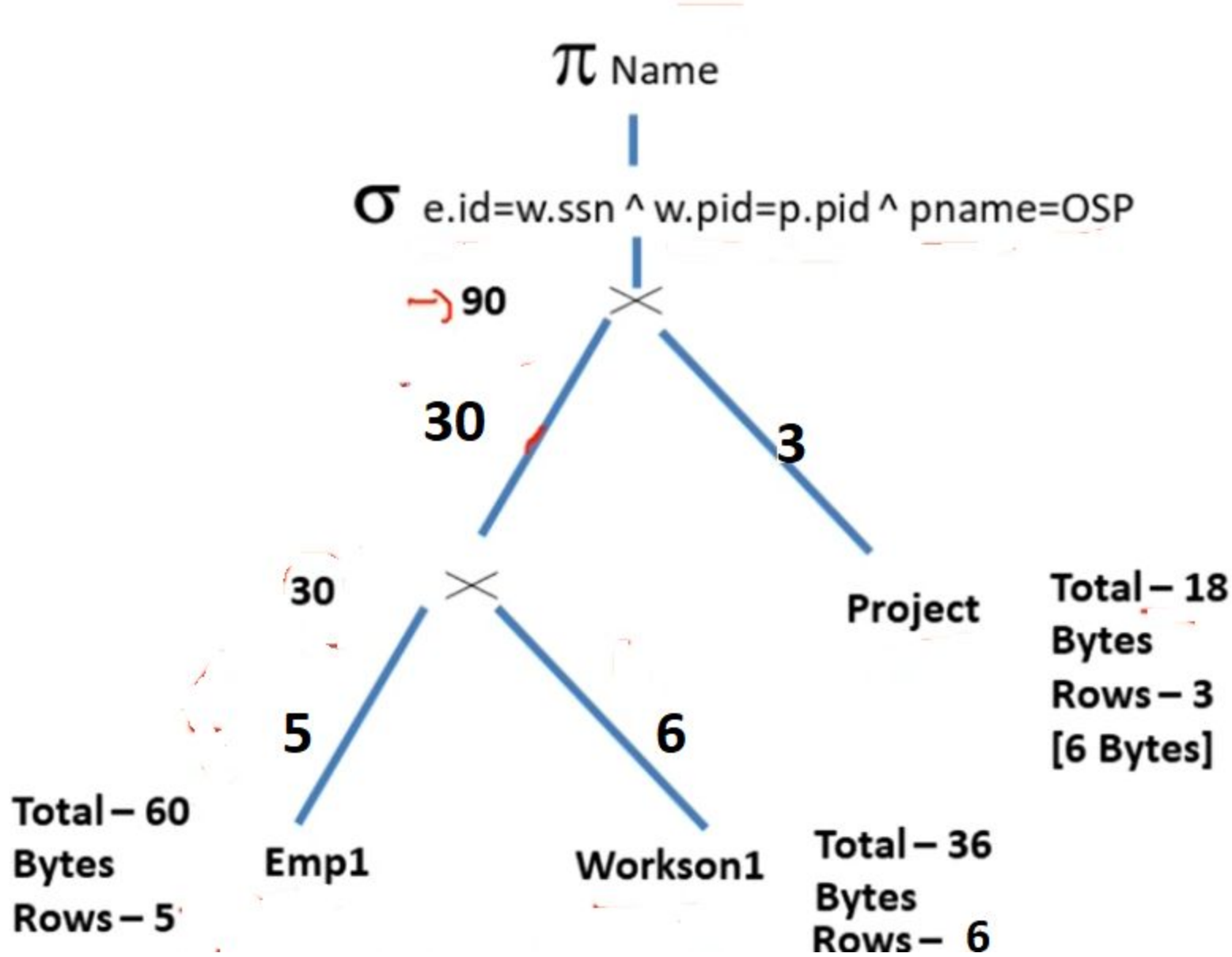
- Construct a Heuristic query optimization for the following query

- Find the names of all the employees who work on the project ‘OSP’
- EMP(EMPID, NAME, DEPT, DESIGN)
- WORKSON(SSN, PID)
- PROJECT(PID,PNAME)

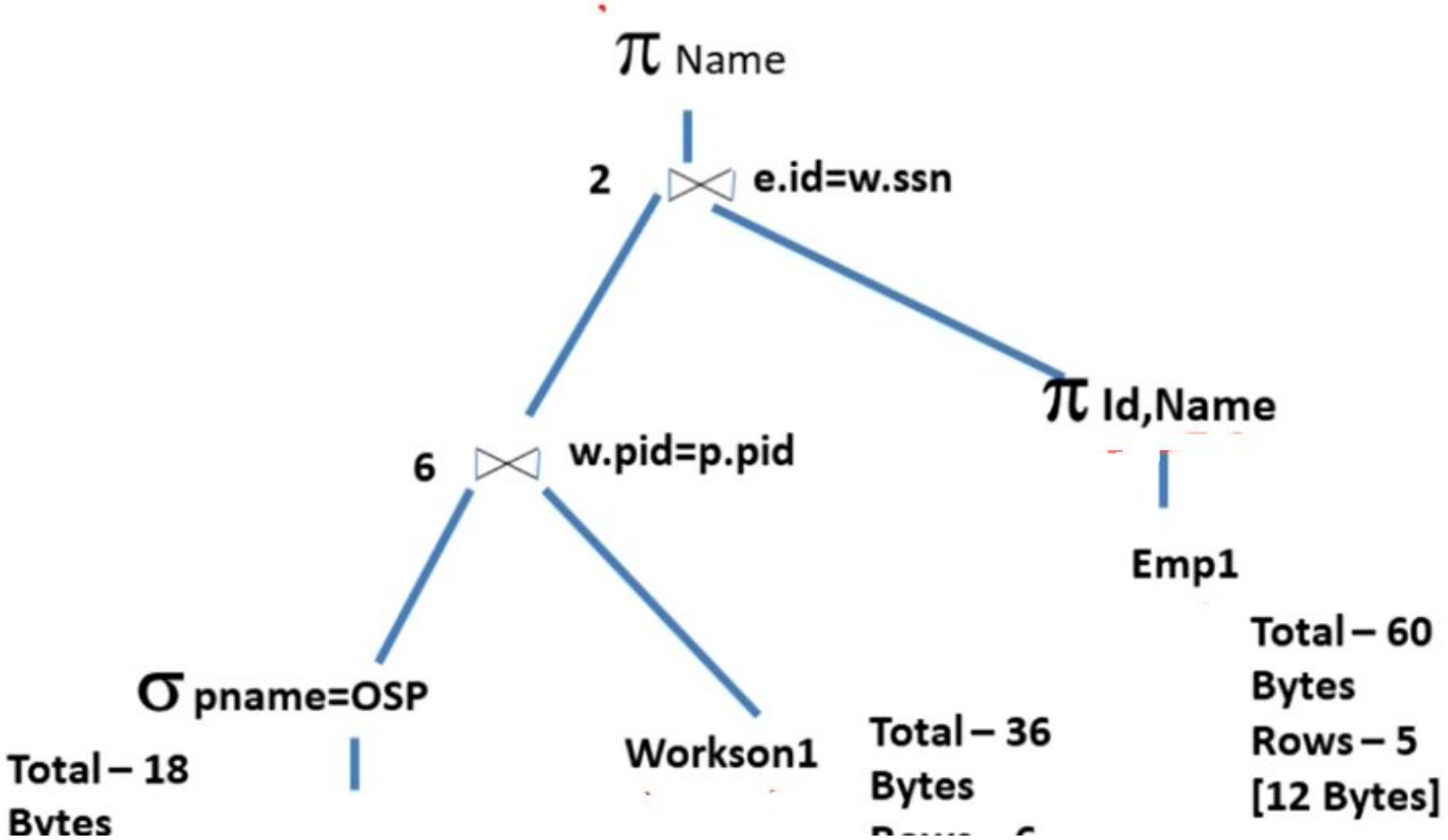
Initial Query Tree



Initial Query Tree



FINAL RESULTANT TREE



Heuristic Optimization of Query Trees-Example

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

Figure 5.5
Schema diagram for the COMPANY relational database schema.

Heuristic Optimization of Query Trees-Example

- **Query**

  "Find the last names of employees born after 1957 who work on a project named ‘Aquarius’."

- **SQL**

  SELECT LNAME

  FROM EMPLOYEE, WORKS_ON, PROJECT

  WHERE PNAME=‘Aquarius’ **AND** PNUMBER=PNO **AND** ESSN=SSN **AND** BDATE >‘1957-12-31’;

Steps in converting a query tree during heuristic optimization.

a) Initial (canonical) query tree for SQL query Q.

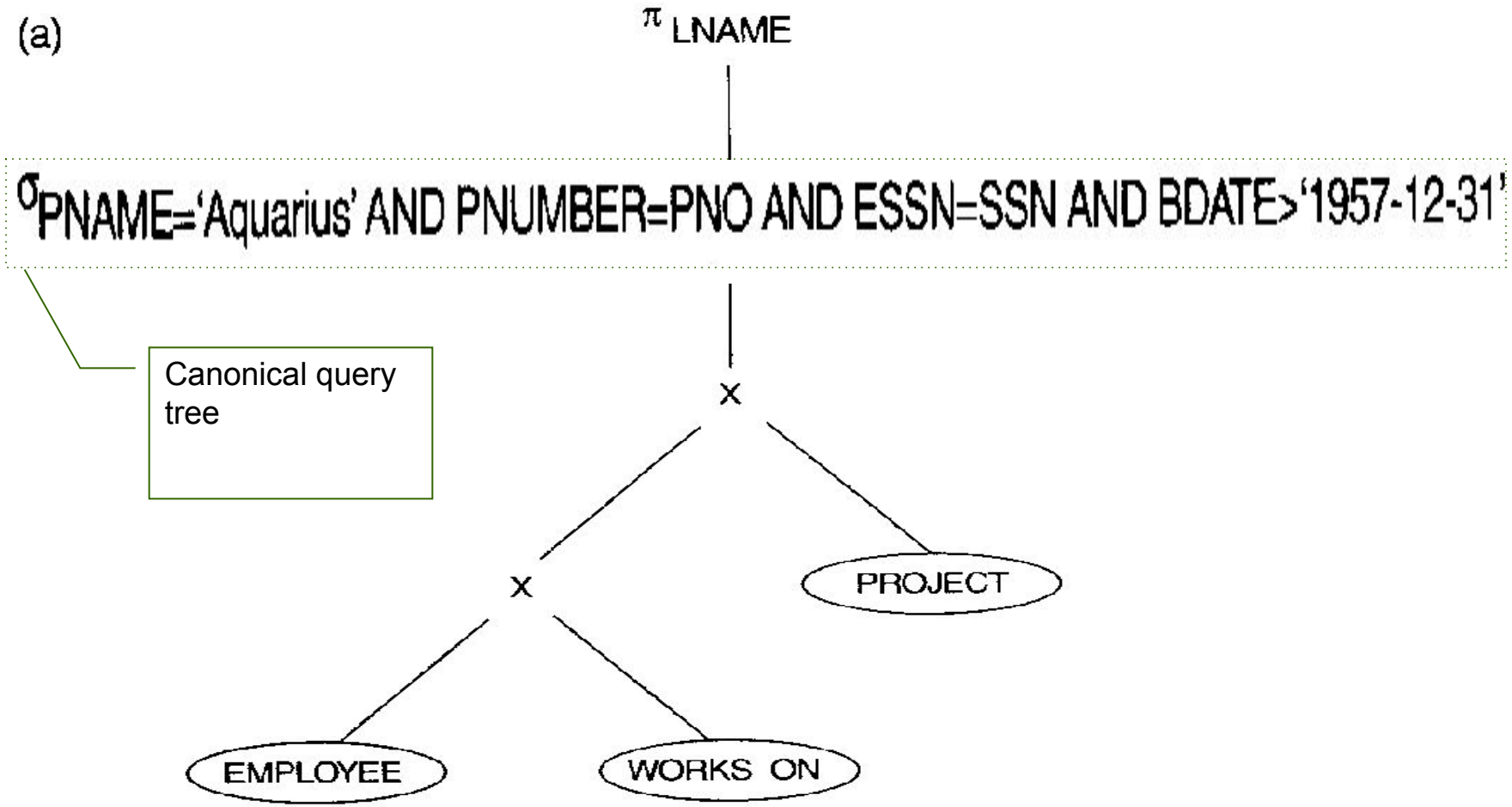
b) Moving SELECT operations down the query tree.

c) Applying the more restrictive SELECT operation first.

d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.

e) Moving PROJECT operations down the query tree.

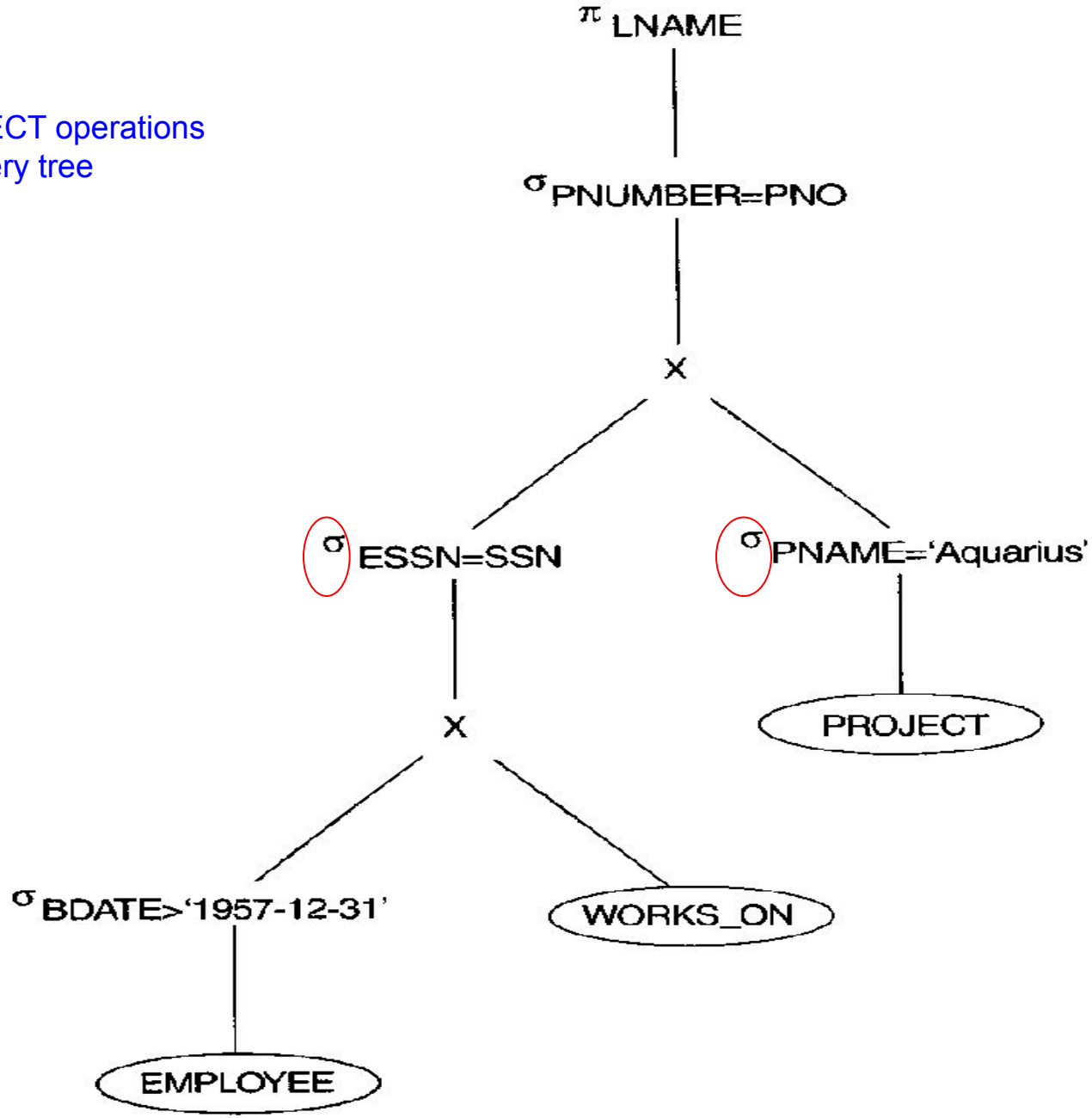
```
SELECT   LNAME
FROM     EMPLOYEE, WORKS_ON, PROJECT
WHERE    PNAME="Aquarius' AND PNUMBER=PNO
         AND ESSN=SSN AND BDATE > 'DEC-31-1957'
```



a) Executing this tree directly first creates a very large file containing the CARTESIAN PRODUCT of the entire EMPLOYEE, WORKS_ON, and PROJECT files. That is why the initial query tree is never executed, but is transformed into another equivalent tree that is efficient to execute. This particular query needs only one record from the PROJECT relation— for the 'Aquarius' project—and only the EMPLOYEE records for those whose date of birth is after '1957-12-31'.

(b)

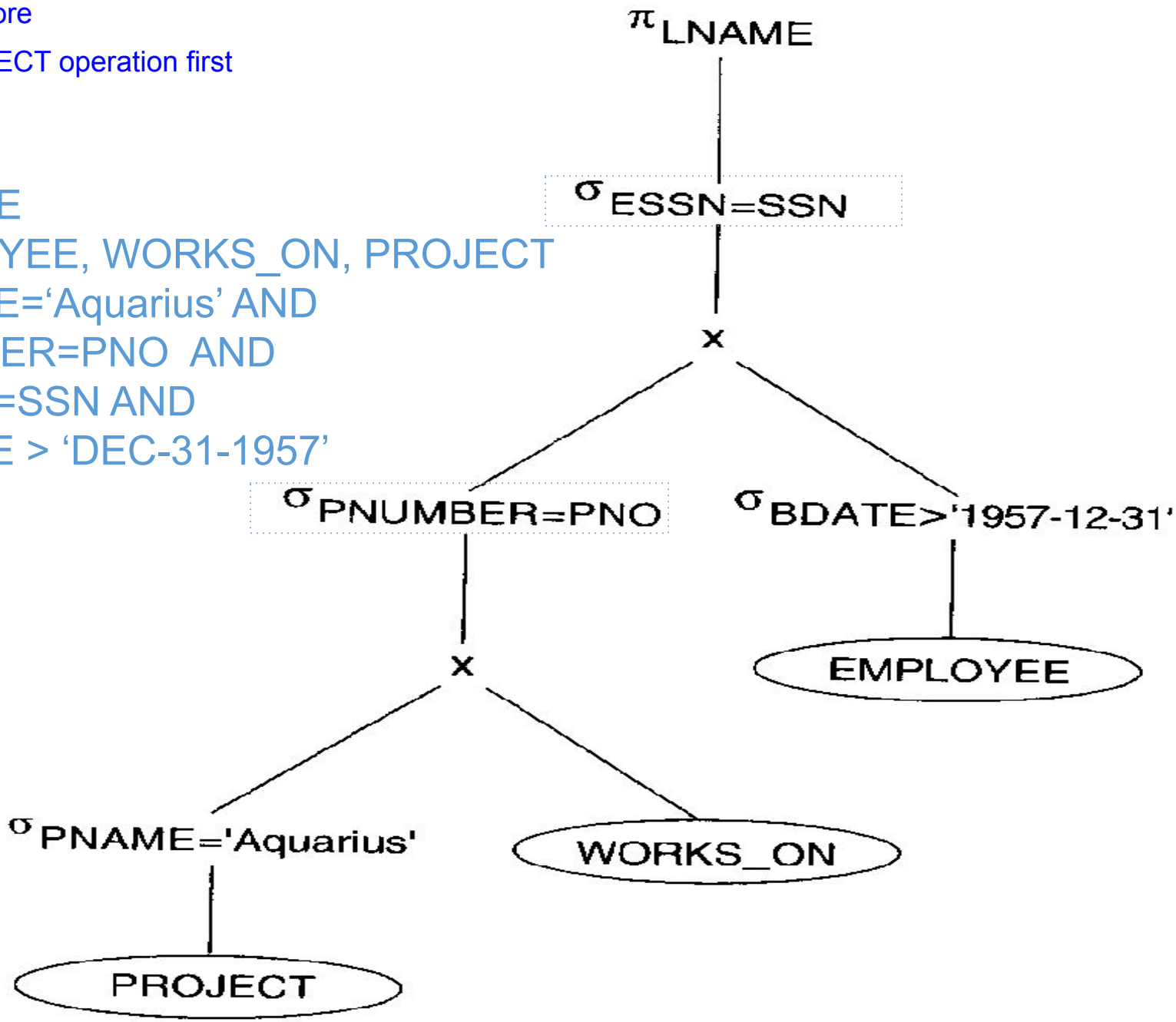
Moving SELECT operations down the query tree



(b) shows an improved query tree that first applies the SELECT operations to reduce the number of tuples that appear in the CARTESIAN PRODUCT.

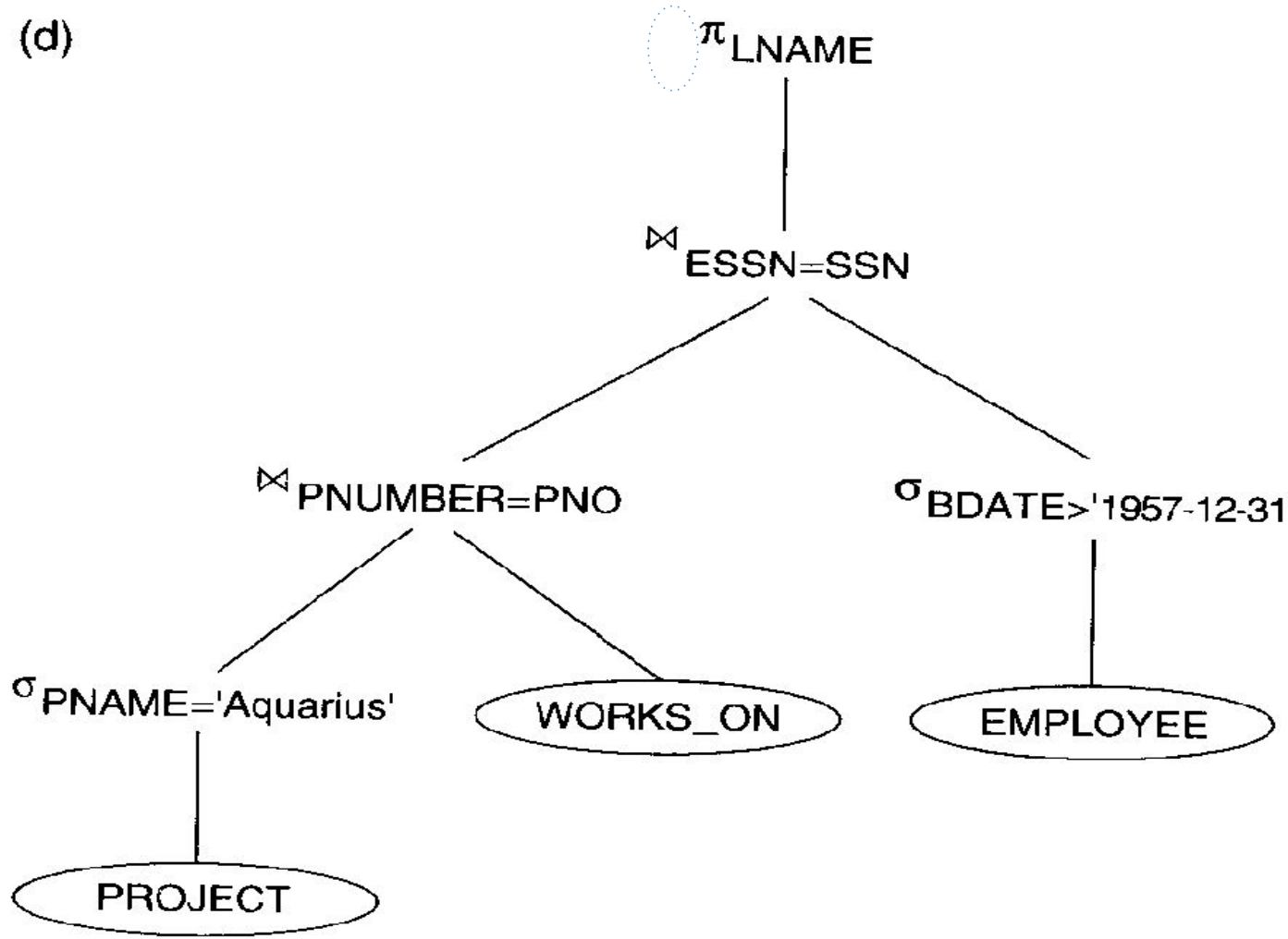
(c) Applying more restrictive SELECT operation first

```
SELECT LNAME
FROM   EMPOYEE, WORKS_ON, PROJECT
WHERE PNAME='Aquarius' AND
          PUMBER=PNO  AND
          ESSN=SSN AND
          BDATE > 'DEC-31-1957'
```



c) A further improvement is achieved by switching the positions of the EMPLOYEE and PROJECT relations in the tree, as shown in Figure (c). This uses the information that Pnumber is a key attribute of the PROJECT relation, and hence the SELECT operation on the PROJECT relation will retrieve a single record only.

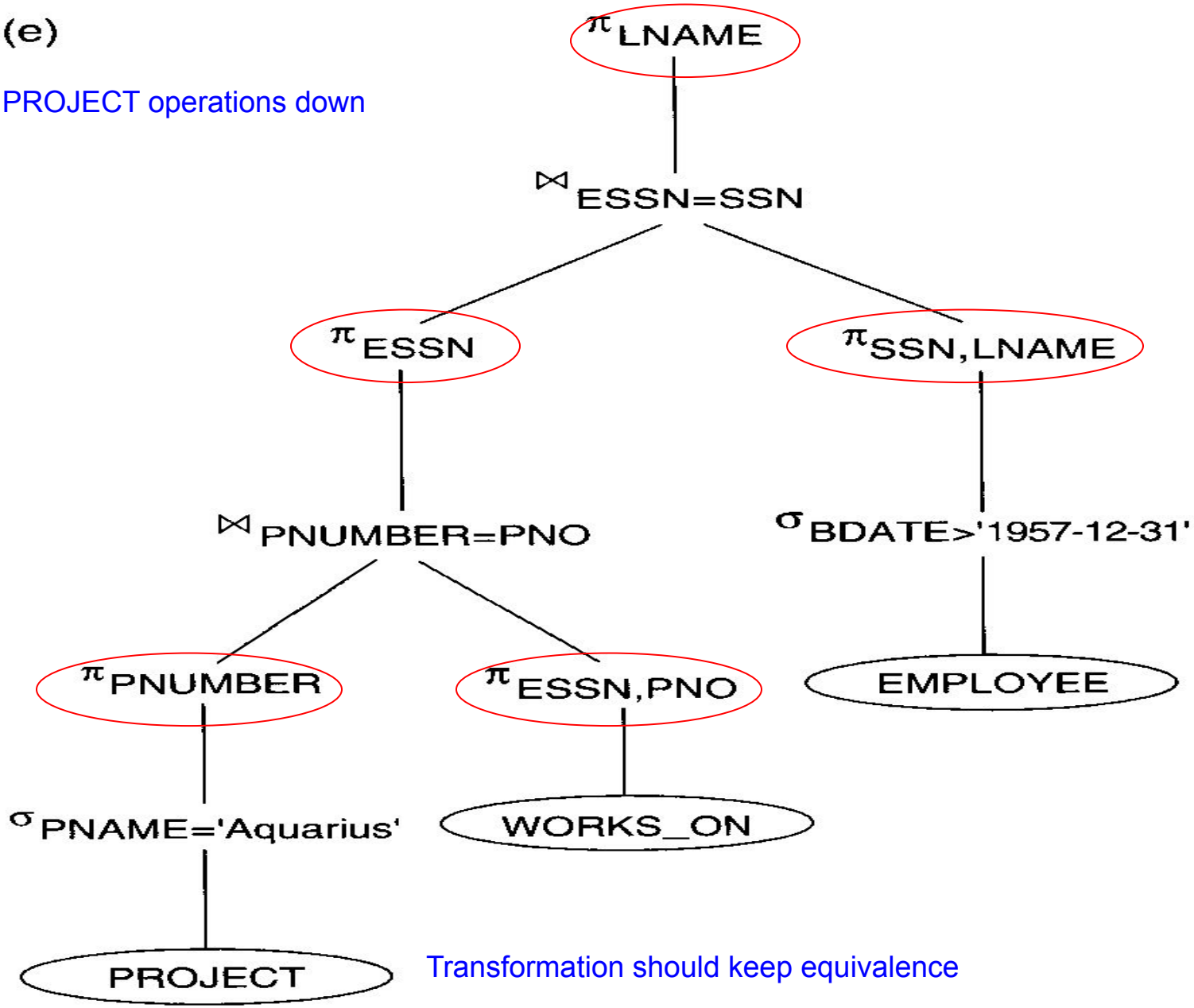
Replacing CARTESIAN PRODUCT and SELECT with JOIN



d) We can further improve the query tree by replacing any CARTESIAN PRODUCT operation that is followed by a join condition with a JOIN operation, as shown in Figure(d).

(e)

Moving PROJECT operations down



Transformation should keep equivalence

e) Another improvement is to keep only the attributes needed by subsequent operations in the intermediate relations, by including PROJECT (π) operations as early as possible in the query tree, as shown in Figure (e). This reduces the attributes (columns) of the intermediate relations, whereas the SELECT operations reduce the number of tuples (records).

SQL Query with an Uncorrelated Subquery

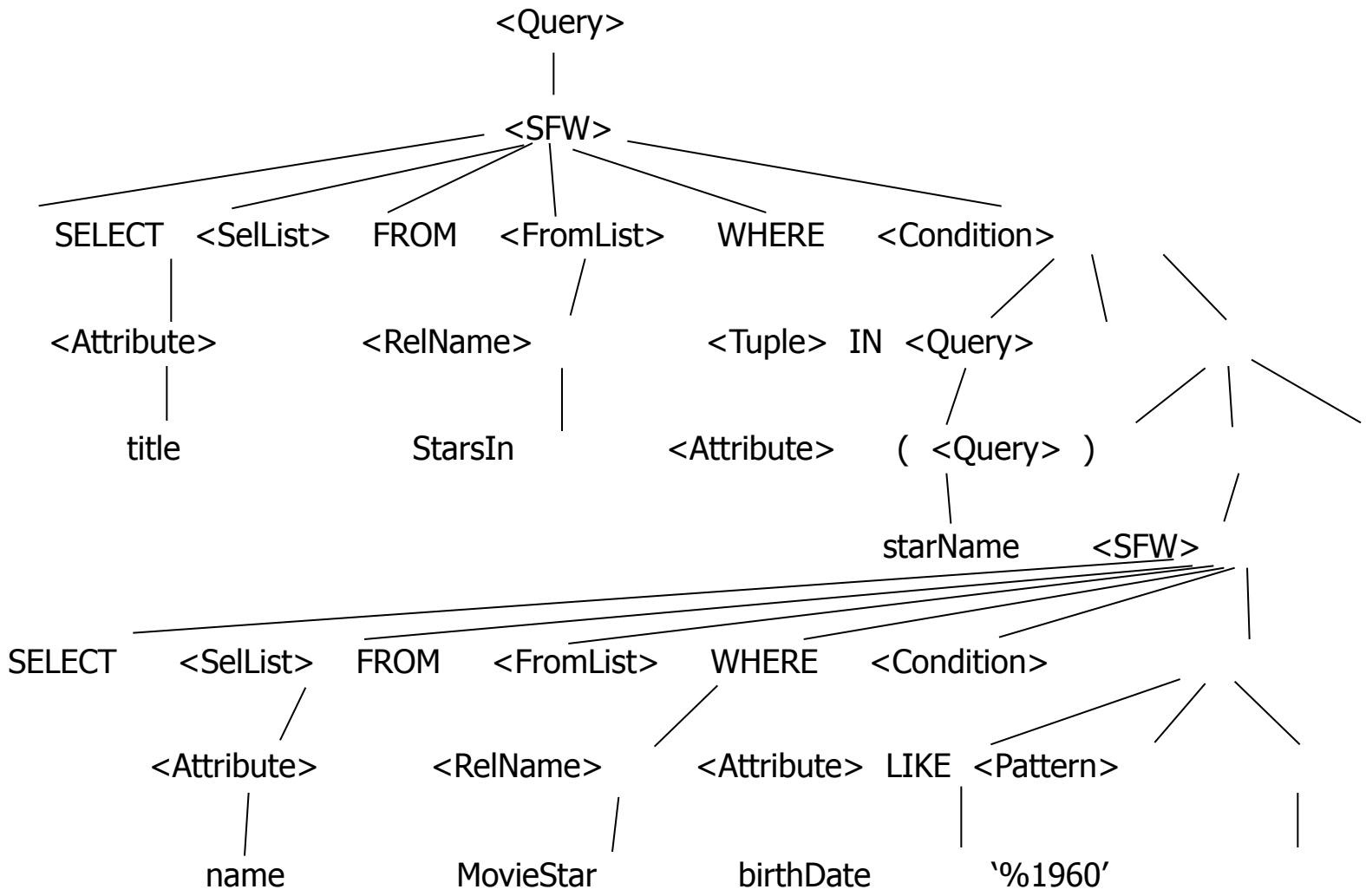
Find the movies with stars born in 1960

MovieStar(name, address, gender, birthdate)

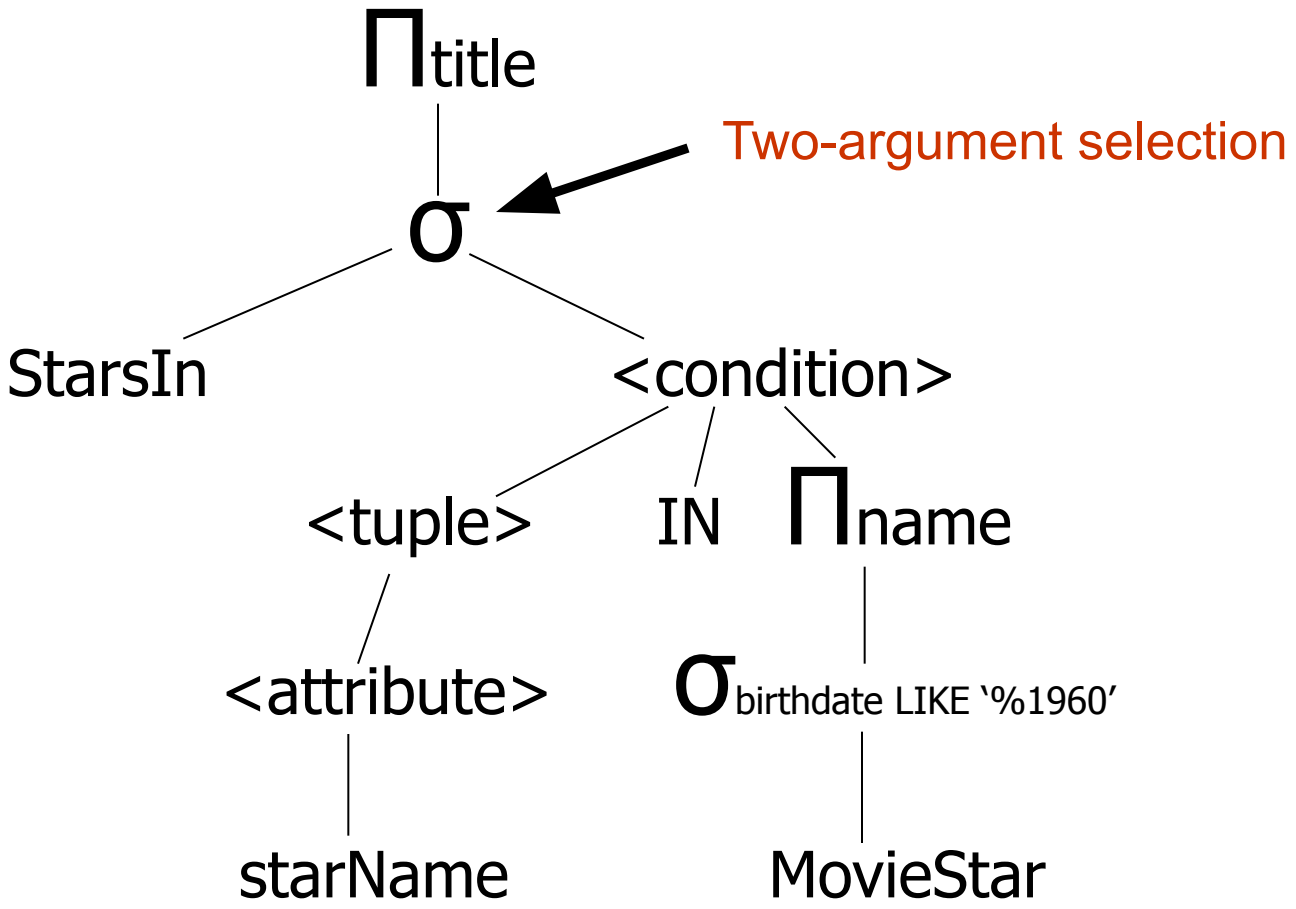
StarsIn(title, year, starName)

```
SELECT title
FROM StarsIn
WHERE starName IN (
      SELECT name
      FROM MovieStar
      WHERE birthdate LIKE '%1960'
);
```

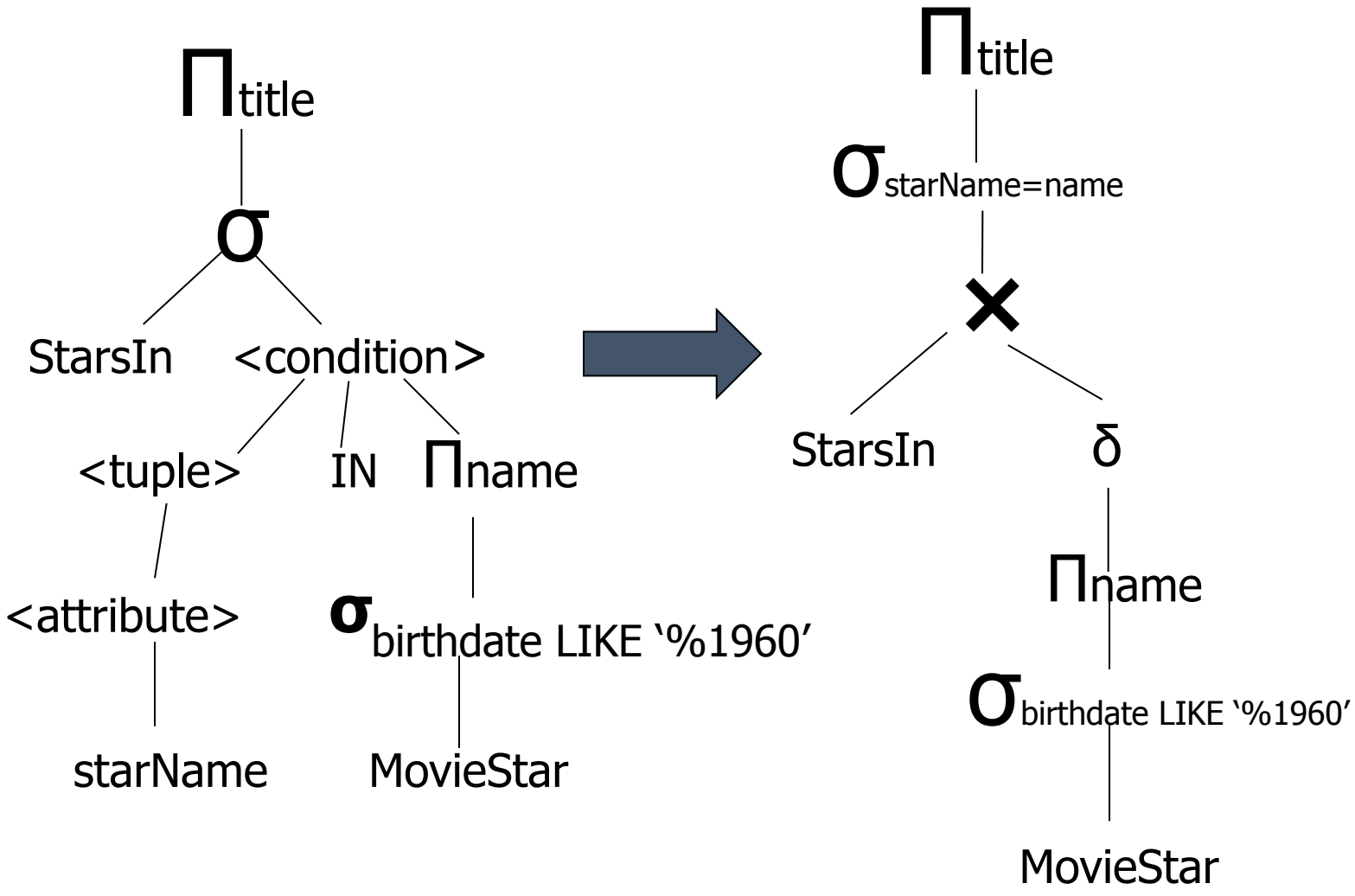
Parse Tree


<Query>
<SFW>
SELECT
<SelList>
FROM
<FromList>
WHERE
<Condition>
<Attribute>
<RelName>
<Tuple>
IN
<Query>
title
StarsIn
<Attribute>
(
<Query>
)
starName
<SFW>
SELECT
<SelList>
FROM
<FromList>
WHERE
<Condition>
<Attribute>
<RelName>
<Attribute>
LIKE
<Pattern>
name
MovieStar
birthDate
'%1960'

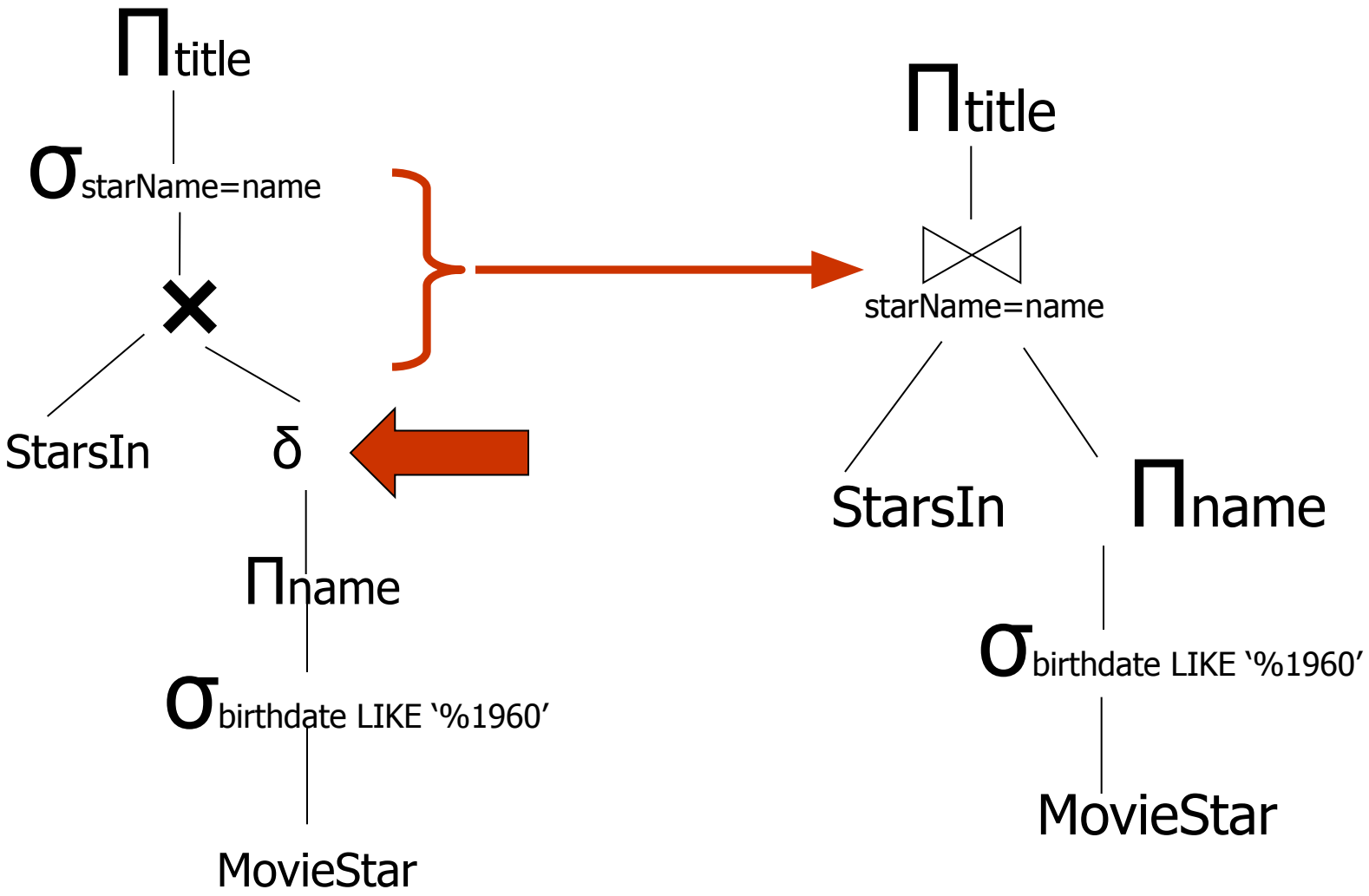
Generating Relational Algebra



Applying the Rewrite Rule



Improving the Logical Query Plan



Thank You